CS101

82 : Networking and the Internet : Protection and Cures

1. Firewall :

A firewall is a network security device or system that monitors and controls incoming and outgoing network traffic, based on a set of security rules.

Firewall also used to protect individual computers rather than entire network or domain.

For Example if a computer is not being used as a web server , an email server , or a name sever , then a firewall should be installed at that computer to block all incoming traffic address to such applications. **A properly installed firewall could block the message from these malicious clients.**

Some Variation of the firewalls are designed specific purposes – an example being **spam filters.**

2. Spoofing :

Spoofing is a cyber security term referring to the practice of impersonating a legitimate entity or device to deceive others.

In Simple Word Spoofing is the act of pretending to be someone or something else to gain unauthorized access or trick a system or person.

3. Proxy Server :

A proxy server is an intermediary server that sits between a user's device and the internet. It receives user requests, forwards them to the destination server, and then returns the response back to the user. It can be used for privacy, security, content filtering, and caching

4. Auditing Software :

Auditing software is an administrator's primary tool for identifying problems before they grow out of control.

Simple word Auditing software helps computer systems **keep a record** of who did what, when, and where — so that you can find out if something went wrong, who caused it, and how to fix it.

5. Antivirus Software :

Another means of defense against invasions via network connections is software called **antivirus software**, which is **used** to detect and remove the presence of known viruses and other infections.

I hope you enjoyed these notes. I have written them in the simplest way for better understanding. Keep me in your prayers. Stay safe and Allah Hafiz

83 : Networking and the Internet : Encryption

Encryption :

A prime example in the secure version of HTTP known as **HTTPS,** which is used to most financial institution to provide customers with secure internet access to their accounts the backbone of HTTP is protocol system known as **Secure Sockets Layer (SSL) ,** which was originally developed by **Netscape** to provide secure communication links between **web clients and server.**

1. Public Key Encryption :

Public key encryption, also known as asymmetric encryption, is a cryptographic technique that uses a pair of keys: a public key and a private key.

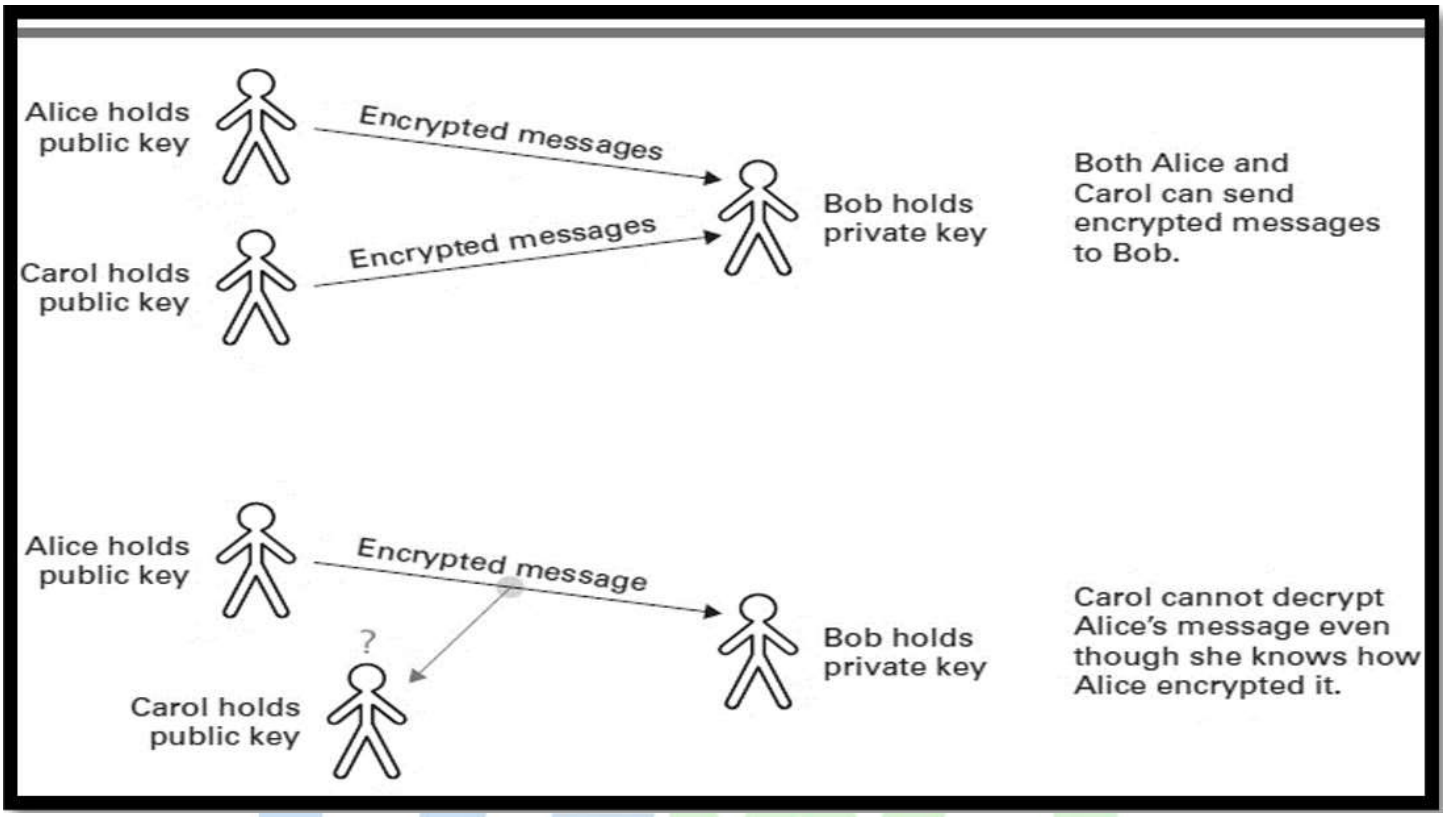
- **Public Key**: Shared with everyone. Used to encrypt messages.
- **Private Key**: Kept secret. Used to decrypt messages.

2. Certificate Authorities :

One Approach to resolving this problem is to established trusted internet sites, called **certificates authorities.**

A **certificates** is a package containing a party's name and that party's public key.

LGWT Learning & Growing With Team



Please Understand This Figure

Module 84

84. Networking and the Internet: Legal Approaches to Network Security

- ✓ Enhancing the security of computer networking systems is to apply legal remedies.
- ✓ Defining what is legal and what is illegal in different contexts.
- ✓ Illegal actions may lead to punishment.

Two Issues

- ✓ Making an action illegal does not stop the action.
- ✓ International law: Illegal in one country and legal in another country.

US – Computer Fraud and Abuse Act

- ✓ Passed in 1984 and has been revised many times.
- ✓ Covers:
  - Worms and viruses
  - Theft of information
  - Many more related things; anything of value is covered.

Contact ME → Click Here

US – Electronic Communication Privacy Act (ECPA)

✓ Employers / ISP have rights to look into the communications of employees.
✓ Illegal for ISP to make available private information.
✓ Employers can access information, especially when the employer's equipment is used.

Pakistan – Cyber Security Law

✓ National Assembly passed a new Cybercrime Law in 2016.
✓ Punishment of 10 years for first offenders and 20 years for repeat offenders.
Details can be found at:
http://thehill.com/policy/cybersecurity/265285-judges-struggle-with-cyber-crime-punishment

✓ Covers many security concerns, e.g.:

- Hacking
- Phishing
- Malware
- Hacking tools
- Identity theft / fraud
- Electronic theft

Pakistan - Acceptable Security Acts:

- ✓ Prevention of Electronic Crimes Ordinance, 2007
- ✓ Electronic Transaction Ordinance, 2002, 2008
- ✓ Pakistan Telecommunication (Re-organization) Act, 1996
- ✓ Wireless Telegraphy Act, 1933
- ✓ Telegraph Act, 1885
- ✓ Federal Investigation Agency Act, 1974
- ✓ Payments and Electronic Fund Transfers Act, 2007

LGWT
Learning & Growing With Team

Module 85

85. Algorithm: an Informal Review

Algorithm

- ✓ Set of steps that define how a task is performed.
- ✓ Let's start discussing simple Algorithms

Making a Cup of Tea

1. Put water in kettle
2. Boil water
3. Put Tea in water
4. Add Milk
5. Stir
6. Add Sugar
7. Put tea in cup
8. Fetch cup

Convert KM to Meters

1. Take input of KM's
2. Multiply the input with 1000.
3. Display the result

Machine Cycle

As long as the halt instruction has not been executed continue to execute the following steps:

a. Fetch an instruction.

b. Decode the instruction.

c. Execute the instruction.

Algorithm Execution :

Many researchers believe that every activity of the human mind, including imagination, creativity, and decision making, is actually the result of algorithm execution.

Module 86

86. Algorithm: Formal Definition of Algorithm

Formal Definition

An algorithm is an ordered set of unambiguous, executable steps that defines a terminating process.

Order vs sequence

- ✓ Have a well-established structure in terms of the order of their execution
- ✓ Does not mean a sequence – like we do not have in parallel processing.
- ✓ Flip-flops producing their output individually and then together they have a meaning.

Executable

- ✓ Make a list of all the positive integers.
- ✓ Computer scientists use the term effective to capture the concept of being executable.

Unambiguous

The information in the state of the process must be sufficient to determine uniquely and completely the actions required by each step.

Make a pretty cartoon!

Terminating Process

The execution of an algorithm must lead to an end.

But!

- ✓ Monitoring the vital signs of a hospital patient
- ✓ Maintaining an aircraft's altitude in flight.

Module 87

87. Algorithm: Abstract Nature of Algorithms

A story is abstract, or conceptual, in nature; a book is a physical representation of a story.

$$F = (\frac{9}{5})C + 32$$

But it could be represented by the instruction : Multiply the temperature reading in Celsius by 9⁄5 and then add 32 to the product

A program is a represent of an algorithm

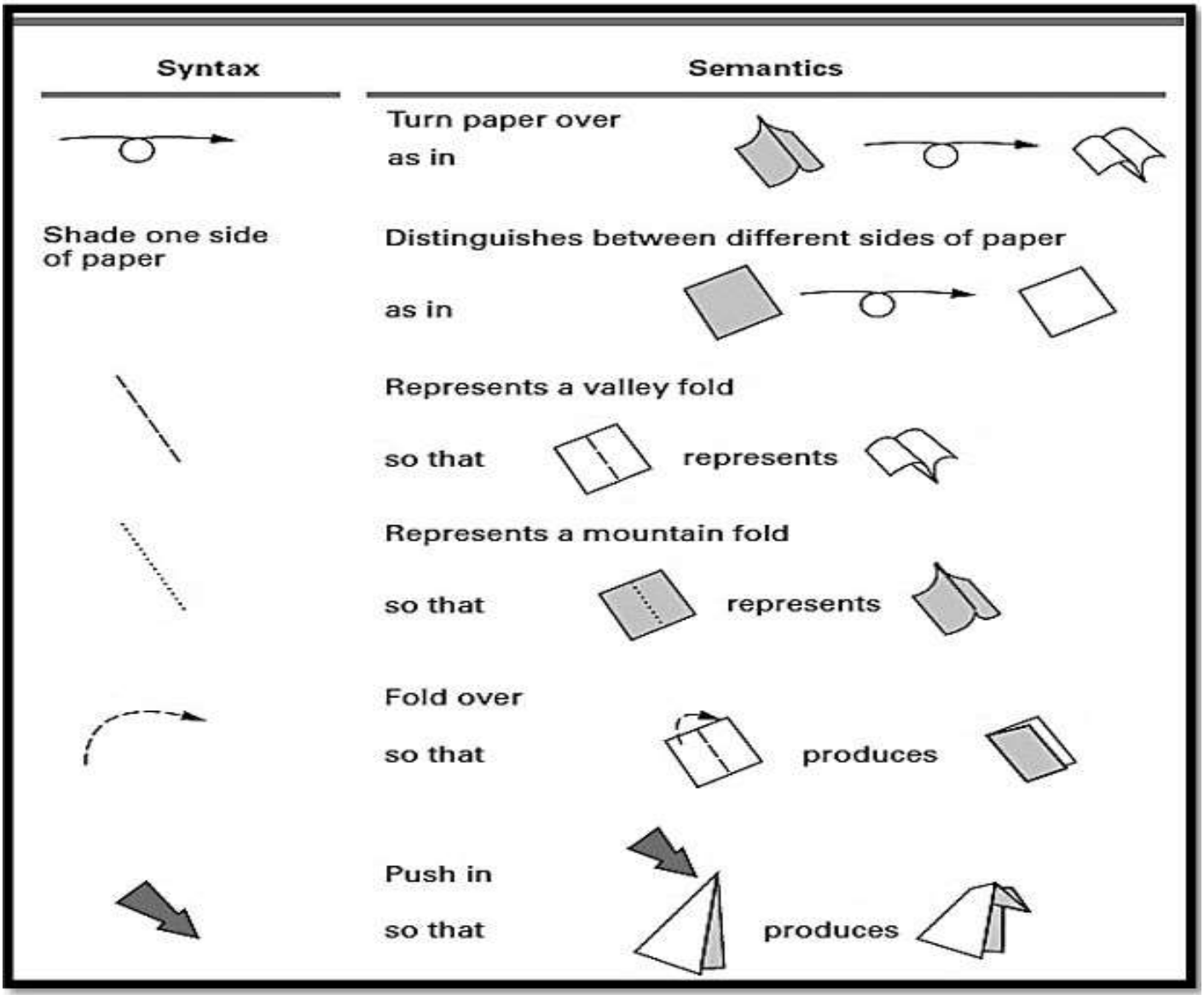
One thing is represent of different methods → But Value Same

Module 88

88. Algorithm: Representation (Primitives)

Primitives:

A well-defined set of building blocks from which algorithm representations can be constructed. Such a building block is called a **primitive.**



Understand This Figure

Module 89

89. Algorithm: Representation (Pseudocode)

Pseudocode:

A pseudocode is a notational system in which ideas can be expressed informally during the algorithm development process.

More recently, pseudocode reminiscent of the Java and C languages has proliferated

For example, the statement

RemainingFunds = CheckingBalance + SavingsBalance

Each of these statements could be rewritten to conform to the structure

if (condition):

activity

else:

activity

It is also called **if-else-if Structure.**

Module 90

90. Algorithm: Representation (Pseudocode) While-Structure

While-Structure:

Another common semantic structure is the repeated execution of a statement or sequence of statements as long as some condition remains true.

For such cases, we adopt the uniform pattern

While(condition):

Activity

LGWT
Learning & Growing With Team

These All Given Only Peseudocode for understanding so please don't fair of this

For Understanding Programing You Can Contact With me If you are not Understand any topics then You Can Discuss With me Thank You!

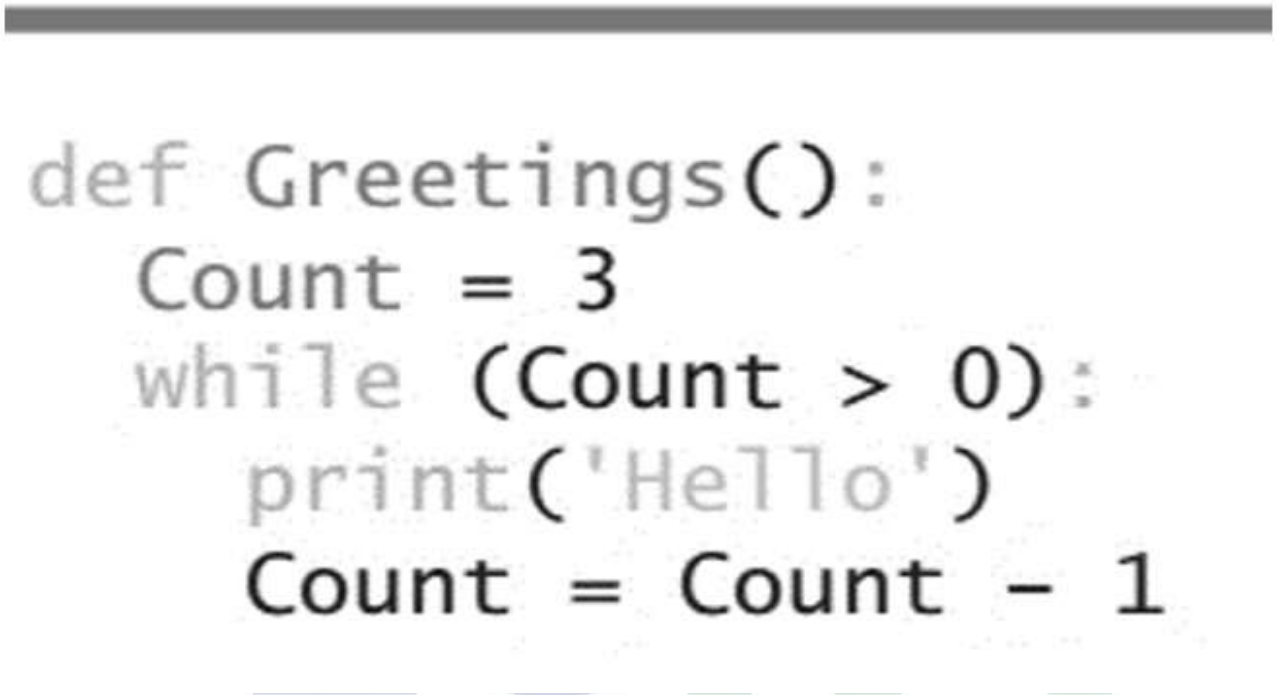
Module 91

Contact ME → Click Here

91. Algorithm: Representation (Pseudocode) Function-Structure

All these Pseudocode are given in python using the term function for our pseudocode and using the Python keyword def to announce the title by which the pseudocode unit will be known.

def name():



For example, if two functions were named **ProcessLoan** and **RejectApplication**, then we could request their services within an if-else structure by writing

if (. . .):

ProcessLoan()

else:

RejectApplication()

Learning & Growing With Team

Module 92

92. Algorithm: Discovery (The Art of Problem Solving)

Problem solving phases presented by the mathematician G. Polya in 1945.

Phase 1. Understand the problem.

Phase 2. Devise a plan for solving the problem.

Phase 3. Carry out the plan.

Phase 4. Evaluate the solution for accuracy and for its potential as a tool for solving other problems.

Contact ME → Click Here

a. Triples whose product is 36		b. Sums of triples from part (a)	
(1,1,36)	(1,6,6)	1 + 1 + 36 = 38	1 + 6 + 6 = 13
(1,2,18)	(2,2,9)	1 + 2 + 18 = 21	2 + 2 + 9 = 13
(1,3,12)	(2,3,6)	1 + 3 + 12 = 16	2 + 3 + 6 = 11
(1,4,9)	(3,3,4)	1 + 4 + 9 = 14	3 + 3 + 4 = 10

Module 93 , 94 , 95 These Topics Not Imported These Topics Are Created Issues So I Advised You That Far From These Topics And Topic 96 Some Important in this Lacture You Can Only Read It One Time And Understand The Figure it If You are not Understand it then Tell Me I Will Guide You Thank You!

Module 96:

```
def Search(List, TargetValue):
  if (List is empty):
    Declare search a failure.
  else:
    Select the first entry in List to be TestEntry.
    while (TargetValue > TestEntry and
        there remain entries to be considered):
      Select the next entry in List as TestEntry.
    if (TargetValue == TestEntry):
      Declare search a success.
    else:
      Declare search a failure.
```

Module 97

97. Algorithm: Iterative Structures (Loop Control)

Loop:

A loop is an instruction that is used to repeatedly execute a code until a certain condition remains true. The block of code that is repeatedly executed each time the loop iterates is called **Body Of Loop**.

C++ Provides the three types of loop.

- ✓ **For Loop**
- ✓ **While Loop**
- ✓ **Do-While Loop**

We Discuss All These Loop Programs In the next Chapters

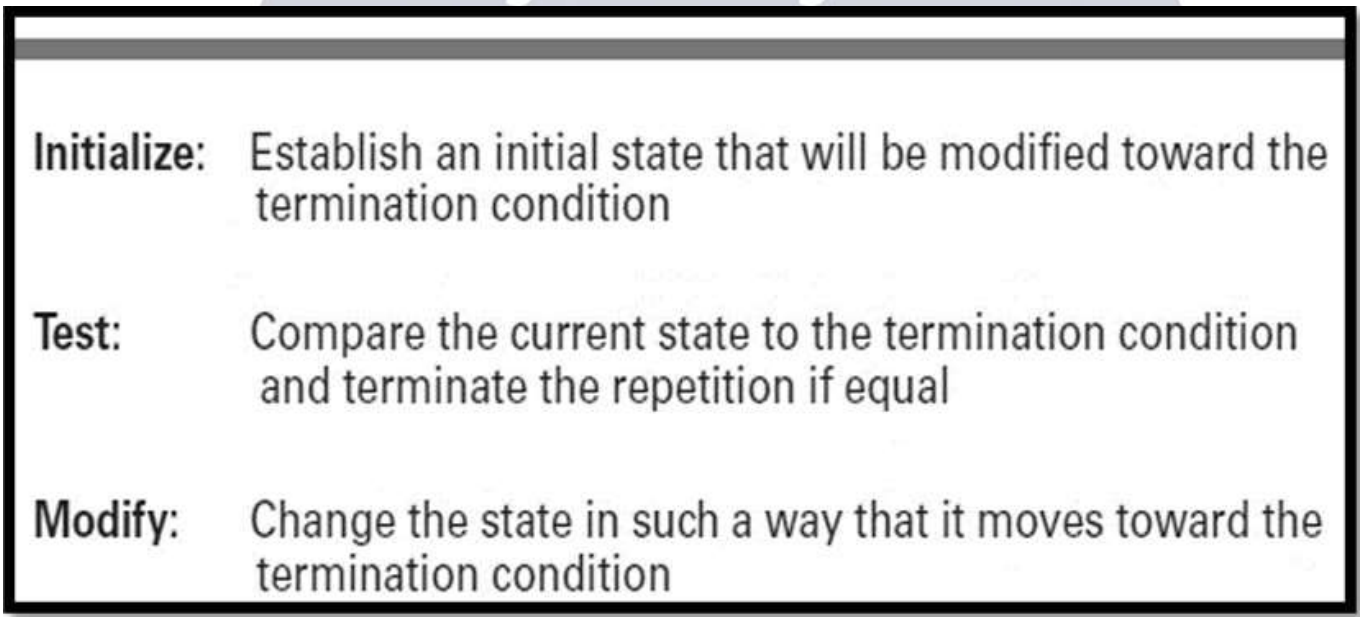
Module 98

98. Algorithm: Iterative Structures (Components of Repetitive Control)

The control of a loop consists of the three activities **initialize, test, and modify.**

Terminating Condition: The condition that is evaluated before a loop starts repeating the statements inside it.

Learning & Growing With Team



Module 99

99. Algorithm: Iterative Structures: Loop Execution (Examples-1)

Let's write a pseudo-code for finding maximum number from a list. Let's assume the following numbers



def FindMax():

max = first number in the list

Current = second number in the list

While (elements in the list exist)

if (max<current)

max=current

Current = next value in the list

Module 100

100. Algorithm: Iterative Structures: Loop Execution (Examples-II)

Let's write a pseudo-code to find a factorial of a number n.

Let's execute it for n=5:

Initially "f" and "i" both will have a value of one. The loop will keep executing until "i" remains smaller or equal to n. In each iteration of "i", the current value of "i" will be multiplied with the current value of f.

At the last when "i" is 6, the loop will terminate as the condition is now false (i<=n)

def FindFactorial():

f = 1

LGWT
Learning & Growing With Team

i=1;

while (i<=n)

f=f*i;

i=i+1;

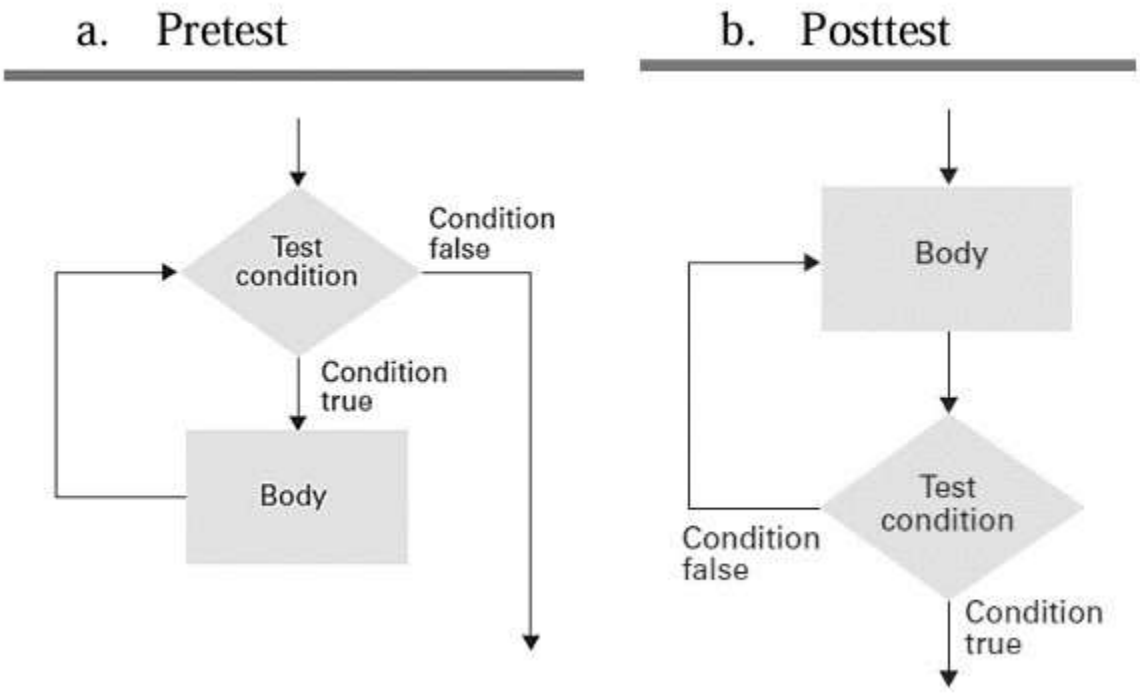
Module 101

101. Algorithm: Iterative Structures (Pretest and Posttest loops)

We will usually refer to these structures as the while loop structure or the repeat loop structure.

Before Executing body the condition is checked is called **pretest.**

After Execution body the condition is checked is called **posttest.**



Module 102 & 103

102. Algorithm: Insertion Sort Algorithm

This means that any numbers of digits or character written as a sequence is called **Sort Algorithm**. We always two things for sorting in any programs with away two things like an example :

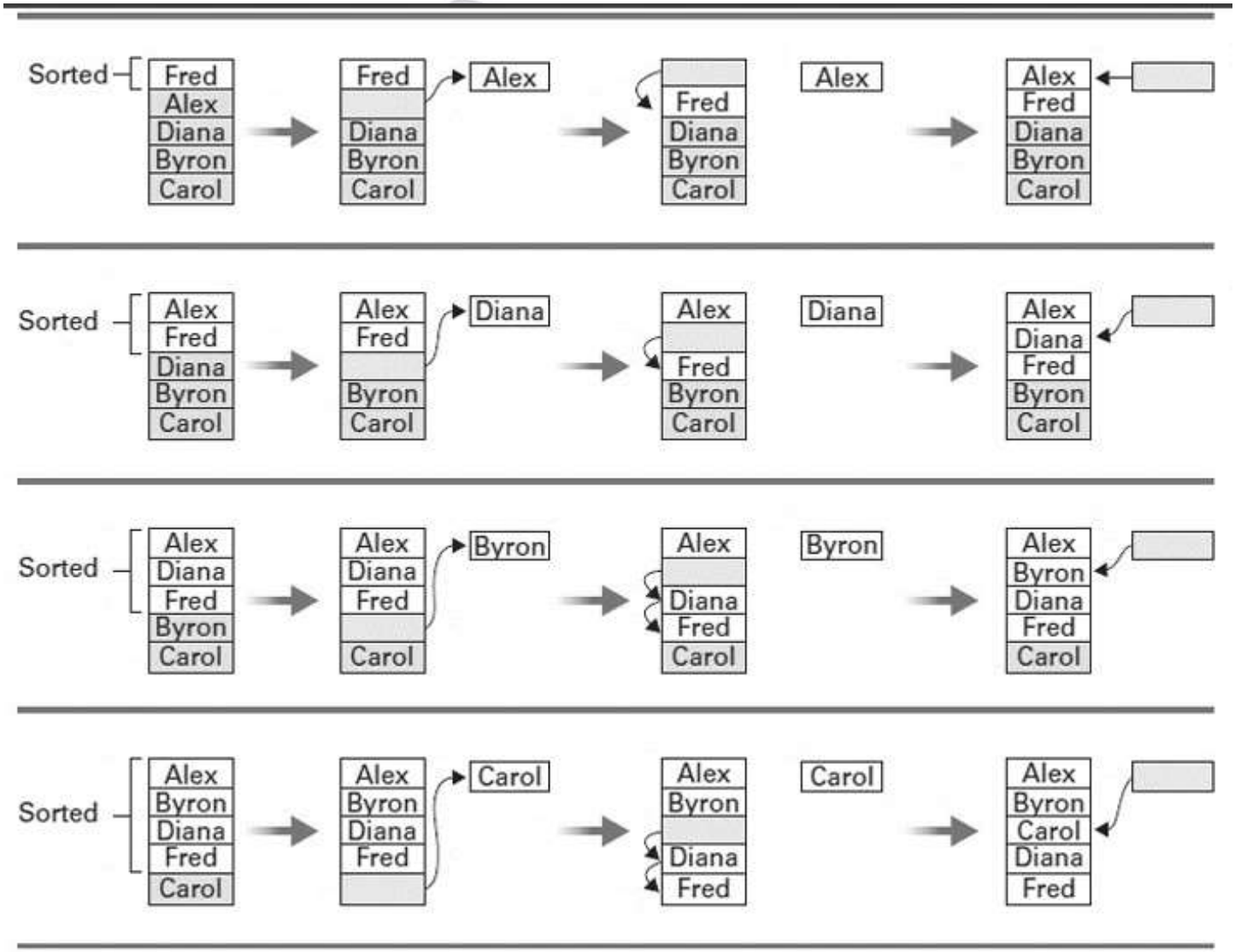
Fred

Alex

Diana

Byron

Carol

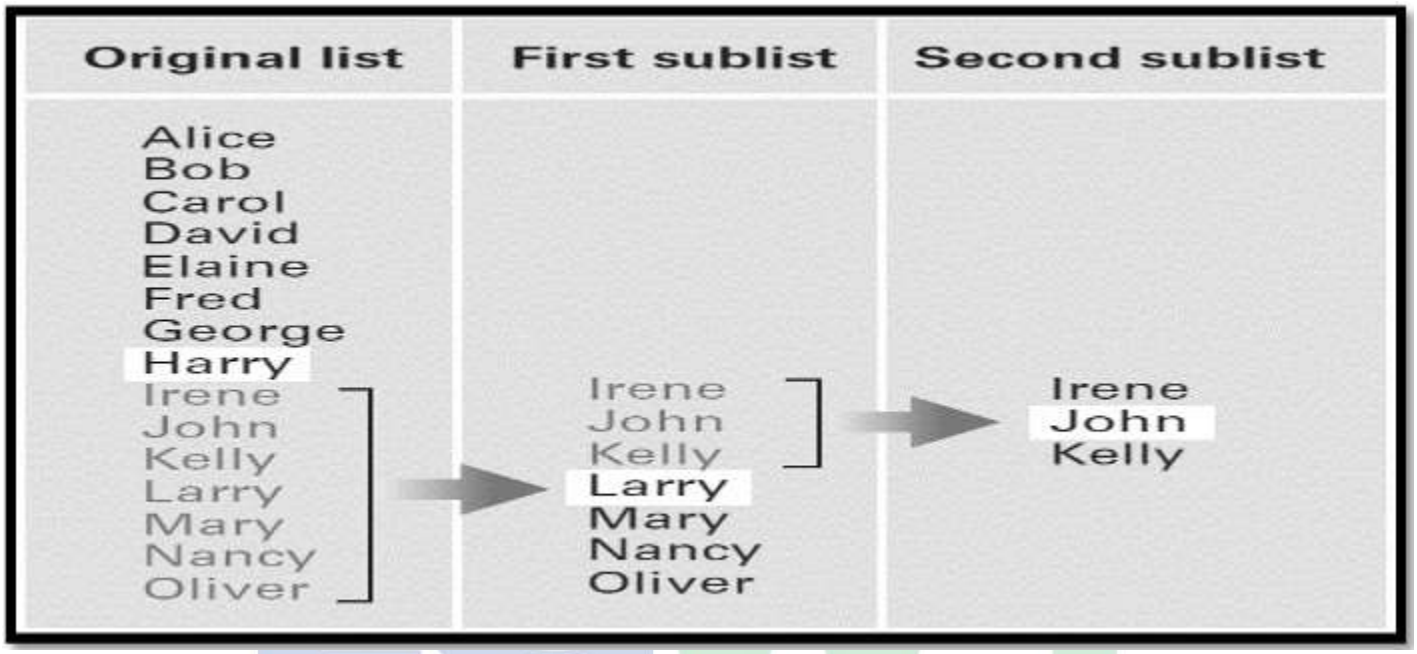


Module 104

104. Algorithm: Recursive Structure (The Binary Search Algorithm)

In binary search we need to follow 3 steps These 3 Steps Are Given Bellow.

First Of all check the **middle** then Computer Order give it middle to **upper** all word check and then **under** the middle down words. **Example in Figure.**



A first draft of the binary search technique.

```
if (List is empty):
  Report that the search failed.
else:
  TestEntry = the "middle" entry in the List
  if (TargetValue == TestEntry):
    Report that the search succeeded.
  if (TargetValue < TestEntry):
    Search() the portion of List preceding TestEntry for TargetValue,
      and report the result of that search.
  if (TargetValue > TestEntry):
    Search() the portion of List following TestEntry for TargetValue,
      and report the result of that search.
```

In the next Figure you can see the pesuedocode

```
def Search(List, TargetValue):
  if (List is empty):
    Report that the search failed.
  else:
    TestEntry = the "middle" entry in List
    if (TargetValue == TestEntry):
      Report that the search succeeded.
    if (TargetValue < TestEntry):
      Sublist = portion of List preceding
        TestEntry
      Search(Sublist, TargetValue)
    if (TargetValue > TestEntry):
      Sublist = portion of List following
        TestEntry
      Search(Sublist, TargetValue)
```

Module 105

105. Algorithm: Recursive Control

Recursive :

The binary search executes each stage of the repetition as a subtask of the previous stage. This technique is known as recursion.

In general, a recursive function is designed to test for the termination condition (often called the base case or degenerative case) before requesting further activations.

Module 105 , 106, 107 are not very important these consist on only examples and history base SO You are see handout and 1 or 2 defines prepare these modules Thank You!

The Module Number 108, 109 And 110 Are Very Very Important So Unerstand it. And keep hard working on it

Contact ME → Click Here

Module 108

108. Algorithm: Software Verification Examples

Software Verification?

- ✓ Distinction between Program believed to be correct and the program that is correct.

Example #1 – Software Failure

- ✓ Fatal Therac-25 X-ray Radiation
- ✓ In 1986, a man in Texas received between 16,500-25,000 radiations in less than 10 sec, over an area of about 1 cm.
- ✓ He passed away 5 months later.
- ✓ The root cause of the incident was a SW failure.

Example #2 – Welsch NHS IT Failure

- ✓ In 2018, doctors and staff were unable to access patients files in UK.
- ✓ This was a technical issue of software not the security issue.
- ✓ Hospitals and GP surgeries were affected.

LGWT
Learning & Growing With Team

Module 109

109. Programming Languages: Early Generations-I

Programs for modern computers consist of sequences of instructions that are encoded as numeric digits. Such an encoding system is known as a **machine language.** Often leads to errors that must be located and corrected (a process known as **debugging**).

In the 1940s, researchers simplified the programing process by developing notational system by which instructions could be represented in mnemonic rather than numeric form.

In Simple Word These Defines.

Machine Language: Machine Language is a low level language that is written in binary form 0 & 1 is called **Machine Language.** Because computer Understand Only machine language.

Debugging: The process of finding and fixing errors, also known as "**bugs**," in computer programs or software.

MOV R5, R6

As a more extensive example, the machine language routine

156C
166D
5056
306E
C000

First Generation

which adds the contents of memory cells 6C and 6D and stores the result at location 6E (Figure 46) might be expressed as

LD
R5,Price
LD R6, ShippingCharge
ADDI R0,R5 R6
ST R0,TotalCost
HLT

using mnemonics. (Here we have used LD, ADDI, ST, and HLT to represent *load, add, store, and halt* . Such descriptive names are often called program **variables or identifiers**.)

LGWT
Learning & Growing With Team

Module 110

110. Programming Languages: Early Generations-II

Once such a mnemonic system was established, programs called **assemblers** were developed to convert mnemonic expressions into machine language instructions.

A mnemonic system for representing programs is collectively called an **assembly language.**

Third generation of programming languages that differed from previous generations in that their primitives were both **higher level and machine independent**. The best known early examples are **FORTRAN** (FORmula TRANslator), which was developed for scientific and engi- neering applications, and **COBOL** (COmmon Business-Oriented Language), which was developed by the U.S. Navy for business applications.

Translator:

A peace of software that handles the translation of source code it into object code.

A program which is input to the compiler is called **Source Code or Source Program.**

The program is know converted to a machine level language by a compiler is know as **Object Code or Object Program.**

Compiler :

A compiler is a software that translate the instruction of high level language to machine language.

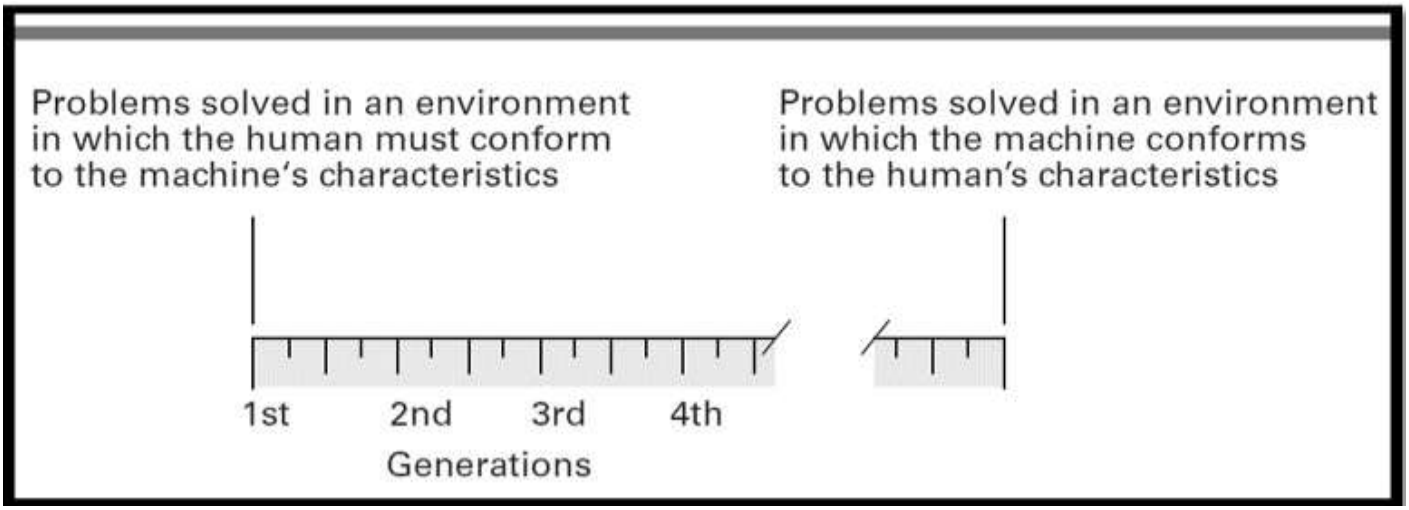
Interpreter :
A program that directly executes the instructions in a high level language with out converting into machine code . **Pyton** is an example of interpreter.

Module 111 is not important so I have skipped this

Module 112

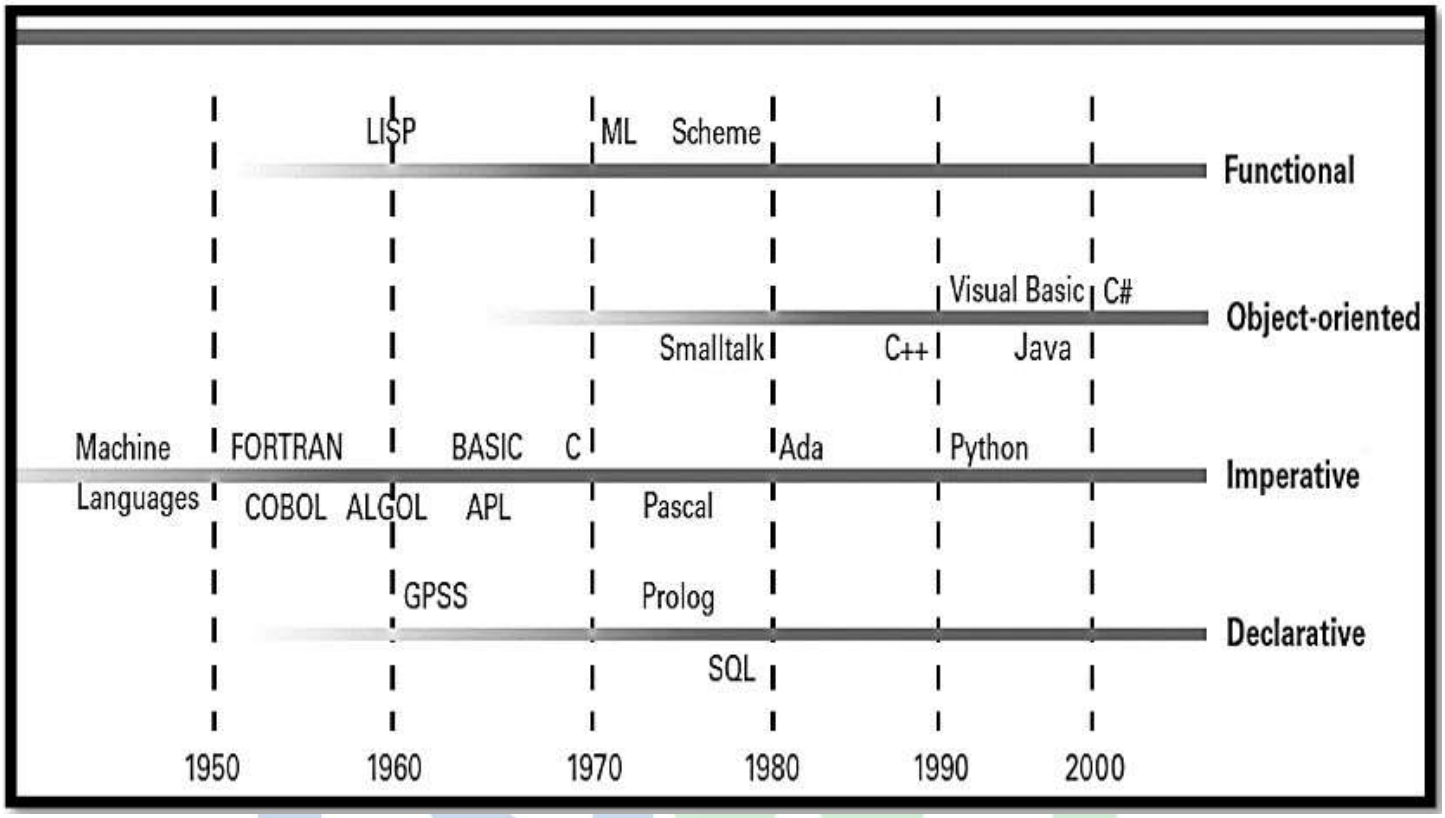
112. Programming Languages: Imperative Paradigms

The different paths as alternative approaches to the programming process (**called programming paradigms**) . The imperative paradigm, also known as the **procedural paradigm** Python and our pseudocode are based as well as the machine language.



Contact ME → Click Here

Must Remember This Table And Understand Because it is very very important



Module 113 & Module 114 Only read One time From Handout

Module 115

115. Programming Languages: Object Oriented Paradigm

Another programming paradigm (and the most prominent one in today's software development) is the **object-oriented paradigm**, which is associated with the programming process called **object-oriented programming (OOP).** Following this paradigm, a software system is viewed as a collection of units, called **objects.**

Each of these objects would encompass a collection of functions (called **methods** in the object oriented vernacular).

This description of the object's properties is called a **class.**

(An object that is based on a particular class is said to be an **instance of that class**.)

In The Next Topics Are started Programs SO Be Carefully focused in it.

Contact ME → Click Here

Module 116

116. Programming Languages: Variable and Data Types

Variable :

A variable is a piece of memory that stores a value that can be changed..

Variable Declaration :

The process of specifying the variable name and its type is called **Variable Declaration.**

Syntax :	Example:
data type variable name	int marks ; float age ;

Variable Initialization:

The process of assign a value to variable is known as variable initialization. The equal sign is used to initialized variable.

Syntax:	Example:
data type variable name = value	int marks = 58; float age = 9.5;

Learning & Growing With Team

Rules of variable declaration :

- The variable may be started with letter or underscore like as (rollno , _level)
- Blank space are not allowed.
- Both upper and lower case are not allowed
- Special symbols are cannot be used as variable name
- A variable can be declare only for one data type
- Reserved word cannot be used as a variable name

Data Type :

A **data type** is an attribute associated with a piece of data that tells a computer system how to interpret its value.

There are 4 data types in this topic

Name	Data Type	Purpose	Example
Integer	int	To store numeric values like (190, 24, 33 e.t.c)	int age = 10;
Float	float	To store real numbers like (7.4 , 68.8 e.t.c)	float marks = 68.5;
Character	char	To store character values like (A, b, c)	char name = 'A';
Boolean	bool	To represents truth values like (True , false)	bool isStudent = true;

Module 117

117. Programming Languages: Data Structure

Data structure, which is the conceptual shape or arrangement of data. One common data structure is the **array**, which is a block of elements of the same type such as a one dimensional list, a two-dimensional table with rows and columns, or tables with higher dimensions.

int Scores[2][9];

An individual element can be identified by means of integer values called **indices.**

In contrast to an array in which all data items are the same type, an **aggregate type** (also called a **structure, a record**, or sometimes **a heterogeneous array**) is a block of data in which different elements can have different types.

struct

{char Name[25];

int Age;

float SkillRating; }

Employee;

LGWT
Learning & Growing With Team

Module 118

118. Programming Languages: Assignment Statement

The most basic imperative statement is the **assignment statement**, which requests that a value be assigned to a variable.

Z = X + Y ;

In some other languages (such as Ada & Pascal) the equiva- lent statement would appear as:

Z := X + Y;

Module 119

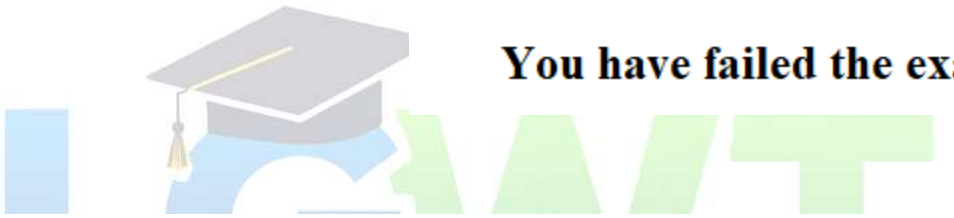
119. Programming Languages:

Control Structures (if-statement) A control statement alters the execution sequence of the program. Of all the programming constructs.

```
if (condition)
    StatementA
else
    StatementB
```

Example:

```
if (marks>=50)
    You have passed the examination
else
    You have failed the examination
```



Module 120

120. Programming Languages: Control Structures (if-statement examples)

In this module, we will learn another example. Suppose a university wants to give a scholarship, if a student gets more than 3.0 CGPA in a given semester. We can write the program in the following way and you have also seen its implementation in the videos in the online compiler.

```
float f=3.5;
if (CGPA>=3.0)
cout<<"Give Scholarship";
else
cout<<"Sorry you do not qualify for the scholarship";
```

Module 120

120. Programming Languages: Control Structure (Loops)

There is another type of control structure known as loop. The loop control structure iterates a set of instructions based on the provided condition.

Consider the following example, we are interested to print the counting from 1 to 5. One way of doing this is as follows:

```
cout<<"1";

cout<<"2";

cout<<"3";

cout<<"4";

cout<<"5";
```

One can write the loop as follows:

```
 int i=1;

while(i<=5) {

cout<<i;

i++;

}
```

This Loop Will executes five times

Module 122

122. Programming Languages: Programming Concurrent Activities

Such simultaneous execution of multiple activations is called **parallel processing or concurrent processing.**

In Simple Wording : One CPU and different purpose executes at a time is called **parallel processing.**

For example, what we have informally referred to as an activation is called a **task** in the Ada vernacular and a thread in Java.

These Are Most Important notes For Final terms Preparation. I hope these all notes are best capable for your study.

Contact ME → Click Here

Module 123

123. Programming Languages: Arithmetic Operators Examples

Arithmetic Operators :

Arithmetic operators is a symbols that performs the mathematical operations on data. C and C++ Languages provides the many arithmetic operators. That's followed here.

Arithmetic Operator Table

Name	Symbols	Example
Addition	**+**	**2 + 2**
Subtraction	**-**	**2 - 2**
Multiplication	*****	**5 * 2**
Division	**/**	**5 / 2**
Modulus	**%**	**5 % 2**

Module 124

124. Programming Languages: Relational Operators Examples

C++ **Relational Operators** are used to compare values of two variables.

<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
= =	Equal to
!=	Not Equal to

Exemple:

```
#include<iostream>
using namespace std;
void main()
{
   int a=10,b=20,c=10;

   if(a>b)
```

LGWT
wing With Team

```
    cout<<"a is greater"<<endl;

  if(a<b)
    cout<<"a is smaller"<<endl;

  if(a<=c)
    cout<<"a is less than/equal to c"<<endl;

  if(a>=c)
    cout<<"a is less than/equal to c"<<endl;
  }
```

Module 125

125. Programming Languages: Logical Operators Examples

C-language Logical Operators are used if we want to compare more than one condition.

Operators	Name of the Operator	Type
&&	AND Operator	Binary
||	OR Operator	Binary
!	NOT Operator	Unary

According to names of the Logical Operators, the condition satisfied in following situation and expected outputs are given

Operator	Output
AND	Output is 1 only when conditions on both sides of Operator become True
OR	Output is 0 only when conditions on both sides of Operator become False
NOT	It gives inverted Output

The Truth Table is given of logical operators

Truth Table

Operator	1st Condition	2nd Condition	Output
AND	True	True	True
	True	False	False
	False	True	False
	False	False	False
OR	True	True	True
	True	False	True
	False	True	True
	False	False	False
NOT	True	–	False
	False	–	True

Module 126

126. Software Engineering: Software Engineering Discipline

Another distinction between software engineering and other engineering disciplines is the lack of quantitative techniques, called **metrics.**

The Main Difference between engineer and software engineer is the metrics.

The application of computer technology to the software development process, resulting in what is called **computer-aided software engineering (CASE)**, is continuing to streamline and otherwise simplify the software development process. CASE has led to the development of a variety of computerized systems, known as **CASE tools.**

Indeed, systems known as **integrated development environments (IDEs)** combine tools for developing software (editors, compilers, debugging tools, and so on) into a single, integrated package.

Module 127

127. Software Engineering: Software Life cycle

What is Software Life Cycle?

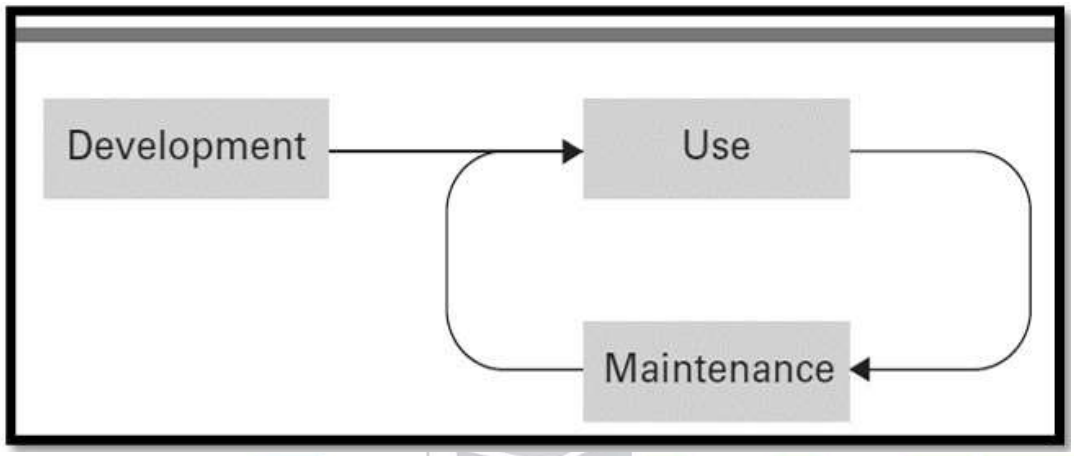
The **Software Life Cycle (SLC)**, also called **Software Development Life Cycle (SDLC)**, is a **step-by-step process** used to **develop, test, and maintain software**. It helps developers and organizations build software in a **systematic and efficient** way.

- Requirement Analysis

Contact ME → Click Here

- Design
- Implementation
- Testing

Software is usually developed once, but it is used and maintained many times throughout its lifetime.



Module 128

128. Software Engineering: Requirement Analysis Phase

Requirement Analysis?

Requirement Analysis is the **first and most important phase** of the Software Development Life Cycle (SDLC).
In this phase, the **software development team meets with the client or users** to understand:

- What the software should do
- What features are needed
- What problems the software will solve

Purpose of Requirement Analysis:

1. ✅ **Understand the client's needs clearly**
2. ✅ **Avoid misunderstandings or wrong features**
3. ✅ **Create a clear plan for software design and development**

Who is Involved?

- **Client/Customer** – the one who needs the software
- **Software Engineer / Analyst** – the one who listens, asks questions, and writes down the requirements
- Sometimes **end-users** are also involved if they will be using the software daily.

Output of this Phase:

Contact ME → Click Here

At the end of the Requirement Analysis phase, a document is created called **SRS – Software Requirements Specification.**

It includes:

- Functional requirements (what the software will do)
- Non-functional requirements (speed, security, etc.)
- User expectations
- System limitations

COTS (Commercial Off-The-Shelf) – Simple Definition:

COTS software is **ready-made software** that is **already developed and available for sale to the general public**. You don't need to build it from scratch — you just **buy it and use it**.

Examples: Microsoft Word , Adobe Photoshop etc

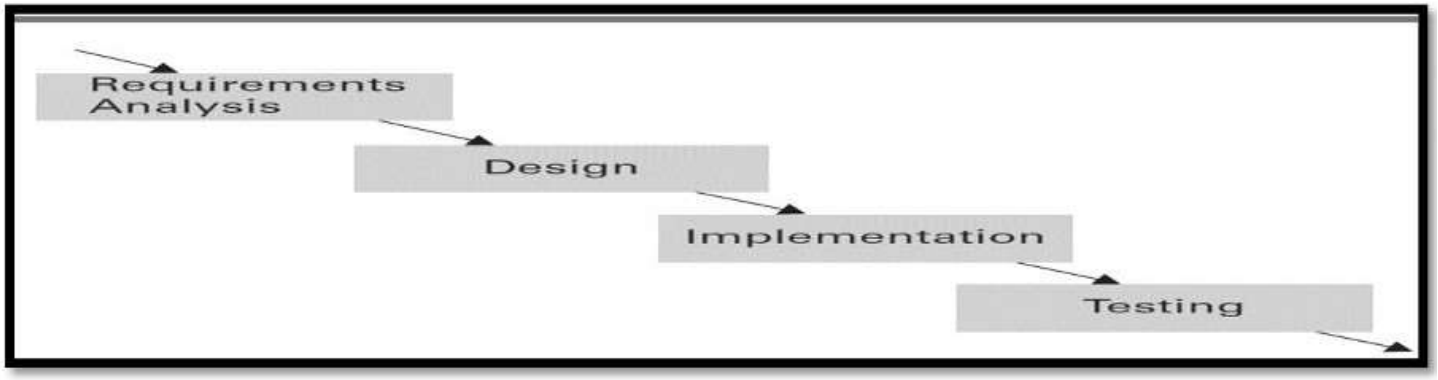


Figure 98: The traditional development phase of the software life cycle

Learning & Growing With Team

Module 129

129. Software Engineering: Design Phase

Design Phase?

The **Design Phase** is the stage where we **plan how the software will work**. It happens **after requirement analysis** and **before coding**.

Purpose:

- To decide **how each part (module)** of the software will be built
- To create a **blueprint** for developers
- To make sure everything is well organized and clear

"To design software, we now use professional and standard ways (like diagrams and models) that have been improved over the years."

Contact ME → Click Here

Module 130

130. Software Engineering: Implementation Phase

Implementation Phase?

The **Implementation Phase** is the stage where the **actual coding (programming)** of the software is done based on the **design**.

- To **convert the software design into working code**
- To **build each module** of the software using a programming language
- To make the software **ready to test and use**

What Happens in This Phase?

- Developers write code according to the design documents
- They use tools like **IDEs, compilers, and libraries**
- Code is written in parts (modules) and later combined (integration)

Module 131

131. Software Engineering: Testing Phase

Testing Phase:

Testing phase is the stage in software development where the software is evaluated to find and fix the bugs or error , insuring that it meets the user requirements and performance correctly.

It producing a three-step development process whose components might have names such as ***"requirements analysis and confirmation," "design and validation,"*** and ***"implementation and testing."***

Process Of Testing :

- **Planning :**
  ➔ Decide what to test , how to test , who will test.
- **Test Cause Design :**
  ➔ Write specific test scenairos with input and expected output.
- **Execution :**
  ➔ Run the software and apply the test causes
- **Bug Reports :**
  ➔ Note Down Any error or issues found during testing
- **Bug Fixing :**
  ➔ Developers correct the reported bugs or problem

LGWT
Learning & Growing With Team

- **Re-Testing :**
  ➔ Test again to make sure the bugs are fixed properly
- **Closure :**
  ➔ Finish Test and prepare the final reports when all major the bugs are fixed.

Simple example:

Sochiye aap ek car bana rahe hain.

- Pehle aap decide karte hain ki car mein kya kya chahiye (requirements).
- Phir car ka design banate hain (design testing).
- Jab parts ban jayein, unka test karte hain (code testing).
- Phir poori car ko chalakar test karte hain (system testing).
- Aakhir mein, car chalakar customer se poochte hain ke wo pasand aayi ya nahi (acceptance testing).

Module 132

132. Software Engineering: Software Engineering Methodologies (I)

Early approaches to software engineering insisted on performing requirements analysis, design, implementation, and testing in a strictly sequential manner.

Software Methodologies :

The different methods of making software is called **Software Methodologies** .

1. **Water – Fall Model :**

The water- fall model, an analogy to the fact that the development process was allowed to flow in only one direction.

In Simple word : water fall model is a software development method where the process flow in one direction.

2. **Incremental Model :**

Incremental model is a development method where the software is built and delivered in small parts.

3. **Iterative Model :**

Iterative model is a method in which software is built repeatedly with improvements made each times. A significant example of iterative techniques is the **rational unified process (RUP)**.

Its popularity has led to the development of a nonproprietary version, called the **unified process**, that is available on a noncommercial basis.

Contact ME ➔ Click Here

In simple wording & Urdu Language

- **Waterfall**: Ek bar mein pura software banate hain, feedback end mein milta hai.
- **Incremental**: Software chhote chhote parts mein banta hai, naye features add karte hain.
- **Iterative**: Wahi software baar baar behtar banate hain, har version mein sudhaar karte hain.

Module 133

133. Software Engineering: Software Engineering Methodologies (II)

Prototyping :

Prototyping in which incomplete versions of the proposed system, called prototypes, are built and evaluated.

Evolutionary prototyping :

In the case of the incremental model these prototypes evolve into the complete, final system, a process known as **evolutionary prototyping**.

Throwaway prototyping :

In a more iterative situation, the prototypes may be discarded in favor of a fresh implementation of the final design. This approach is known as **throwaway prototyping**.

An example that normally falls within this throwaway category is **rapid prototyping.**

Learning & Growing With Team

📌 Final One-Line Summary:

Type	Simple Explanation
Prototyping	Sample banana user ko dikhane ke liye
Evolutionary	Wohi sample baar baar behtar banake final software ban jata hai
Throwaway	Sample sirf dikhane ke liye hota hai, baad mein delete kar ke naya software banate hain

Open source development refers to a collaborative software development approach where the source code is made freely available for anyone to use, modify, and distribute

Agile Methods are ways of developing software in which the software is made in **small parts (steps)**. After making each part, **feedback is taken from the customer**, and if needed, **changes can be made easily**. This method focuses on **fast work**, **teamwork**, and **customer satisfaction**.

One example of an agile method is **extreme programming (XP).**

Module 134
134. Software Engineering: Software Engineering Methodologies (II)

If it were not for **modularity** that is, the division of software into manageable units, generically called **modules.**

Structure chart , in which functions are represented by rectangles and function dependencies (implemented by function calls) are represented by arrows.

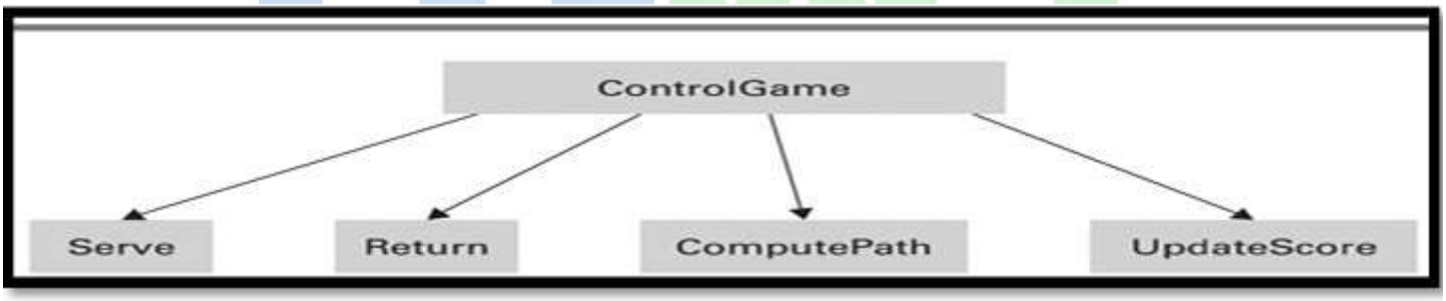


Figure 99: A simple structure chart

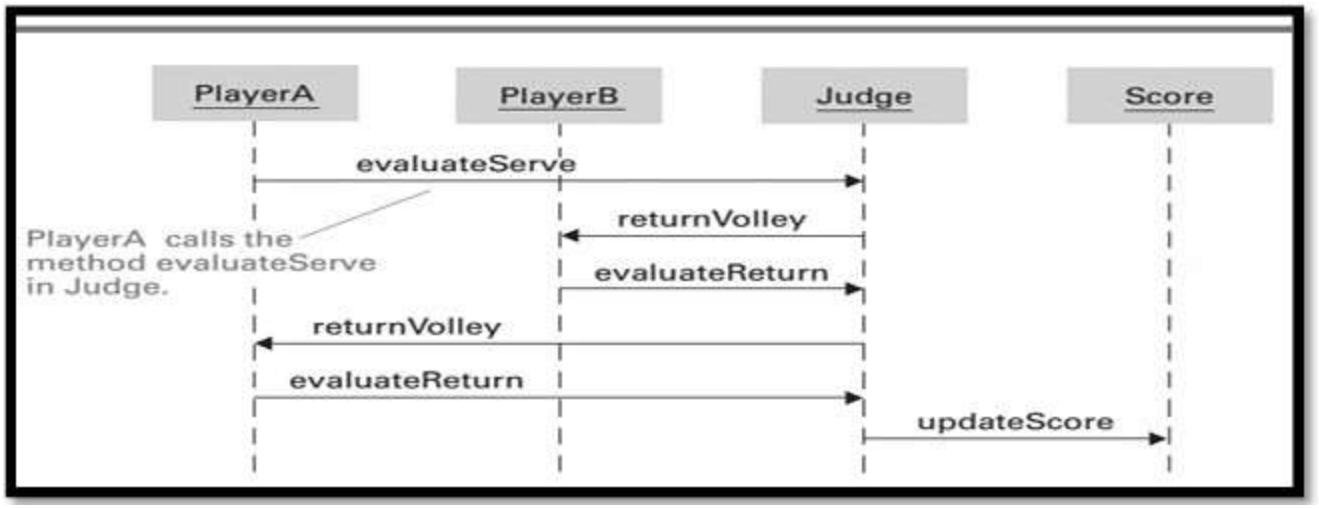
Module 135

135. Software Engineering: Coupling

Consequently, a goal when designing a modular system should be to maximize independence among modules or, in other words, to minimize the linkage between modules (known as intermodule **coupling**).

One is **control coupling**, which occurs when a module passes control of execution to another, as in a function call

Contact ME → Click Here



Another form of intermodule coupling is **data coupling**, which refers to the sharing of data between modules.

In contrast to passing data explicitly as parameters, data can be shared among modules implicitly in the form of **global data.** which are data items that are automatically available to all modules throughout the system

Module 136

136. Software Engineering: Cohesion

The term **cohesion** refers to this internal binding or, in other words, the degree of relatedness of a module's internal parts.

Simple Word : Cohesion means that the tasks being performed within a module serve a single purpose.

Types of Cohesion :

- A weak form of cohesion is known as **logical cohesion.**
- A stronger form of cohesion is known as **functional cohesion**, which means that all the parts of the module are focused on the performance of a single activity

Module 137

137. Software Engineering: Information Hiding

Information Hiding is a software design principle in which **internal details of a module or component are hidden from other parts of the system**, and only the necessary information is exposed.

Contact ME → Click Here

Simple Explanation:

Information hiding means **keeping the inner workings (code, logic, data) of a module private**, and allowing access only through a **public interface**.

Module 138
138. Software Engineering: Components

Component, which is, by definition, a reusable unit of software.

IN simple Word : A software component is a reusable and independent part of a software system that performs a specific function

Component Architecture :

Research in the development and use of components has led to the emerging field known as component architecture.

Component Assembler :

A component assembler is a tool or a person responsible for putting together various software components to build a complete software system.

Module 139

139. Software Engineering: Design Patterns

A **design pattern** is a pre-developed model for solving a recurring problem in software design.

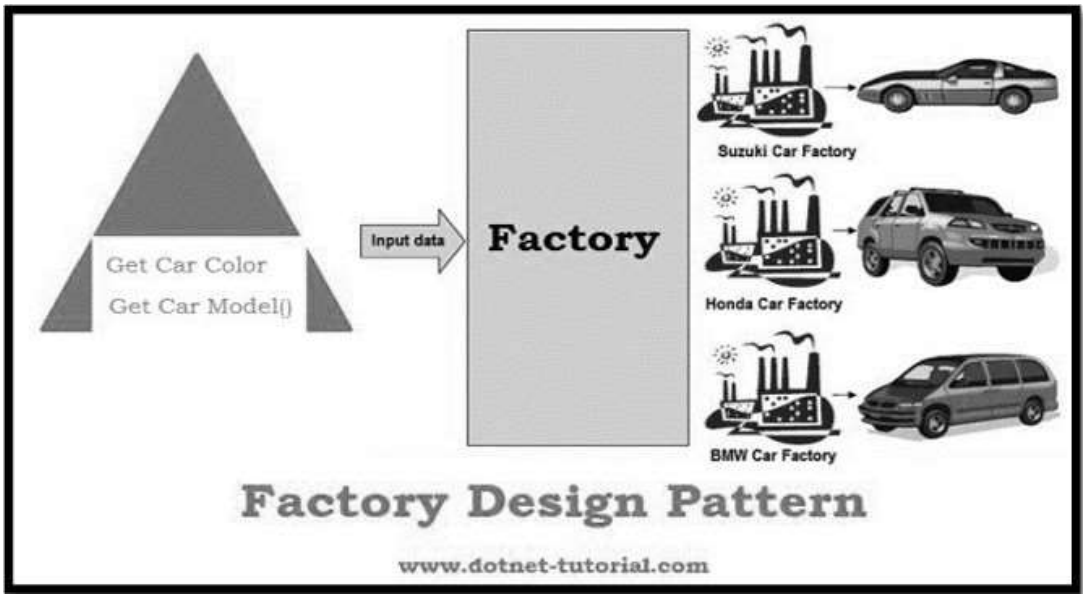
Module 140

140. Software Engineering: Design Patterns Examples

Factory design Pattern:

The Factory Design Pattern (Figure 102) is a commonly used design pattern where we need to create Loosely Coupled System. Basically, it comes under Creational Pattern and it is used to create instance and reuse it. Factory Pattern is based on real time factory concept. As we know, a factory is used to manufacture something as per the requirement and if new items are added in the manufacturing process, the factory starts manufacturing those items as well. Factory class provides abstraction between Client and Car when creating the instance of the Car [Honda, BMW etc] increasingly powerful tool for software engineers is the growing collection of design

Contact ME → Click Here



Shopping Cart Design Pattern:

- Use when the user can possibly buy more than one product.
- Use when the user can possibly buy more than one instance of a product.
- Use when the user may want to return later to carry on shopping
- Use when the user may want to return at a later time to conduct payment
- Do not use when you only have one product to sell.
- Do not use when your site is arranged in a way, so that it does not make sense for the user to buy more than one product at a time (for instance for Application Service Providers (ASPs) allowing a user to upgrade his service).

LGWT
Learning & Growing With Team



Learning & Growing With Team

Module 141

141. Software Engineering: Scope of Quality Assurance

Software Quality Assurance (SQA) is a set of activities for ensuring quality in software engineering processes. It involves monitoring and improving the entire software development process to ensure that the final product meets quality standards and user requirements.

In Simple Words : SQA means checking every step of software development to make sure the software is correct, reliable, and free of errors.

Contact ME → Click Here

Some Explain in Urdu :

*International organizations jaise **ISO, IEEE, ACM** software development ki **quality aur professionalism** ko improve karne ke liye **standards** banate hain, aur companies inhi standards ke mutabiq kaam karne ke liye **SQA teams** banati hain.*

Module 142

142. Software Engineering: Software Testing

Pareto principle:

Pareto principle, in reference to the *economist and sociologist* Vilfredo Pareto **(1848−1923)** who observed that a small part of ***Italy's*** population controlled most of ***Italy's*** wealth.

Another software testing methodology, called **basis path testing**, is to develop a set of test data that insures that each instruction in the software is executed at least once.

Glass-box testing, meaning that the software tester is aware of the interior structure of the software and uses this knowledge when designing the test.

Black-box testing, which refers to tests that do not rely on knowledge of the software's interior composition.

An example of black-box testing is the technique, called **boundary value analysis**, that consists of identifying ranges of data, called **equivalence classes**, over which the software should perform in a similar manner and then testing the software on data close to the edge of those ranges

Beta Testing in which a preliminary version of the software is given to a segment of the intended audience with the goal of learning how the software performs in real-life situations before the final version of the product is solidified and released to the market. (Similar testing performed at the developer's site is called **Alpha Testing.)**

Module 143

143. Software Engineering: Documentation

Software documentation serves three purposes, leading to three categories of documentation: **user documentation**, **system documentation**, and **technical documentation**.

User Documentation : User documentation is to explain the features of the software and describe how to use them. *It Give the User*.

Contact ME → Click Here

System Documentation : system documentation is to describe the software's internal composition so that the software can be maintained later in its life cycle. *It has the developer*.

Technical Documentation : Technical documentation is to describe how a software system should be installed and serviced (such as adjusting operating parameters, installing updates, and reporting problems back to the software's developer).

Module 144

144. Software Engineering: Human Machine Interface

One of the tasks during requirements analysis is to define how the proposed software system will interact with its environment.

Research in human-machine interface design draws heavily from the areas of engineering called **Ergonomics**, which deals with designing systems that harmonize with the physical abilities of humans, and **Cognetics**, which deals with designing systems that harmonize with the mental abilities of humans.

MCQS May Be

The **GOMS** (rhymes with "Toms") model, originally introduced in 1954, is representative of the search for metrics in the field of human-machine interface design.

Module 145

145. Software Engineering: Software Ownership and Liability

Intellectual Property: which is based on the well-established principles of copyright and patent law

Indeed, the purpose of a copyright or patent is to allow the *developer of a product to release that product intended parties while protecting his or her ownership rights.*

Software License: A software license is a legal agreement between the owner and user of a software product *It means Save your Properties* .

Module 146

146. Data Abstraction: Arrays and Aggregates

Array :

An array is a "rectangular" block of data whose entries are of the same type. The simplest form of array is the

Contact ME → Click Here

One-Dimensional Array, a single row of elements with each position identified by an index. A one-dimensional array with **26 elements** could be used to store the number of times.

Two-Dimensional Array consists of multiple rows and columns in which positions are identified by pairs of indices—the ***first index identifies the row*** associated with the position; ***the second index identifies the column***

Aggregate Type is a block of data items that might be of different types and sizes. The items within the block are usually called **Fields.**

Module 147 is very important about MCQS

Module 147

147. Data Abstraction: List, Stacks and Queues

Another basic data structure is a **List**, which is a collection whose entries are arranged sequentially. The beginning of a list is called the **Head Of the List**. The other end of a list is called the **Tail**.

Types of List :

It consist of two Types : 1) **Stack** **2) Queues**

- **A stack** is a list in which entries are inserted and removed only at the head.

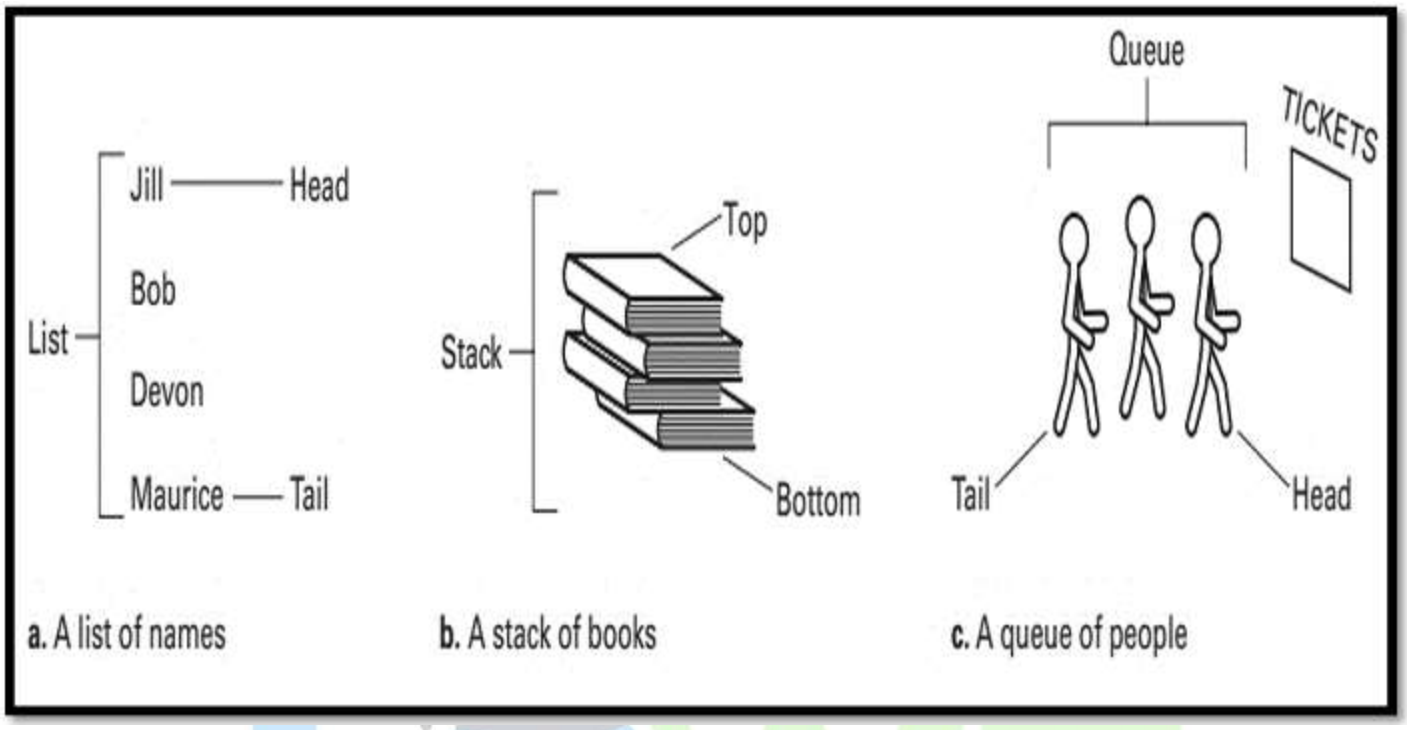
The head of a stack is called the **top of the stack**. The tail of a stack is called its **bottom or base**. Inserting a new entry at the top of a stack is called **pushing an entry**. Removing an entry from the top of a stack is called **popping an entry**. Note that the last entry placed on a stack will always be the first entry removed—an observation that leads to a stack being known as a **Last-In, First-Out**, or **LIFO (pronounced "LIE-foe")** structure.

Learning & Growing With Team

The term **backtracking** refers to the process of backing out of a system in the opposite order from which the system was entered.

- **A queue** is a list in which the entries are removed only at the head and new entries are inserted only at the tail.

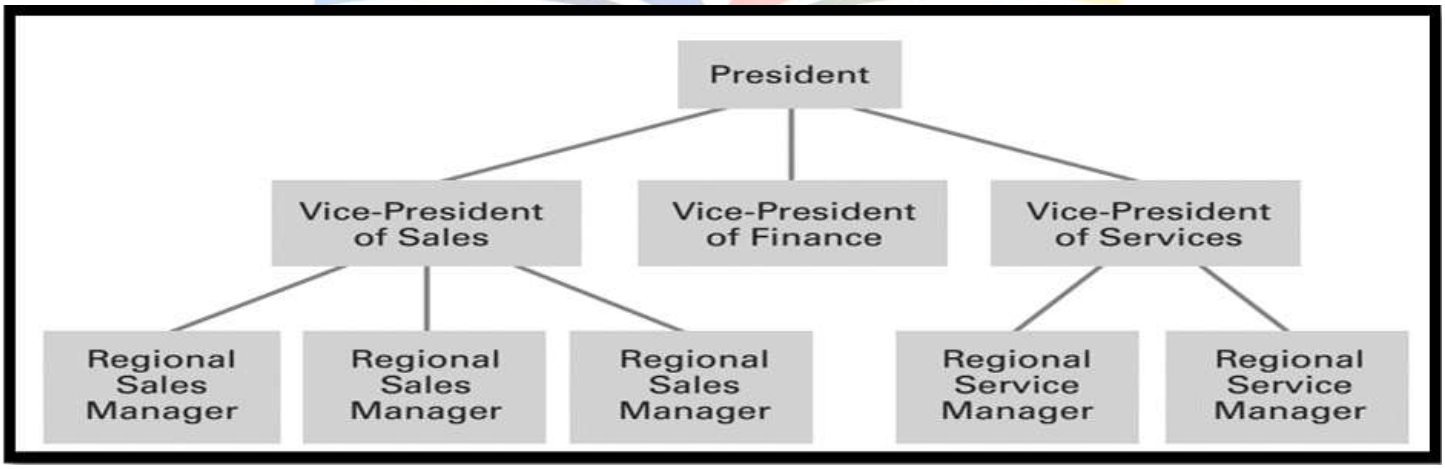
A queue is a **First-In, First-Out**, or **FIFO** (pronounced "FIE-foe") structure, meaning that the entries are removed from a queue in the order in which they were stored.



Module 148

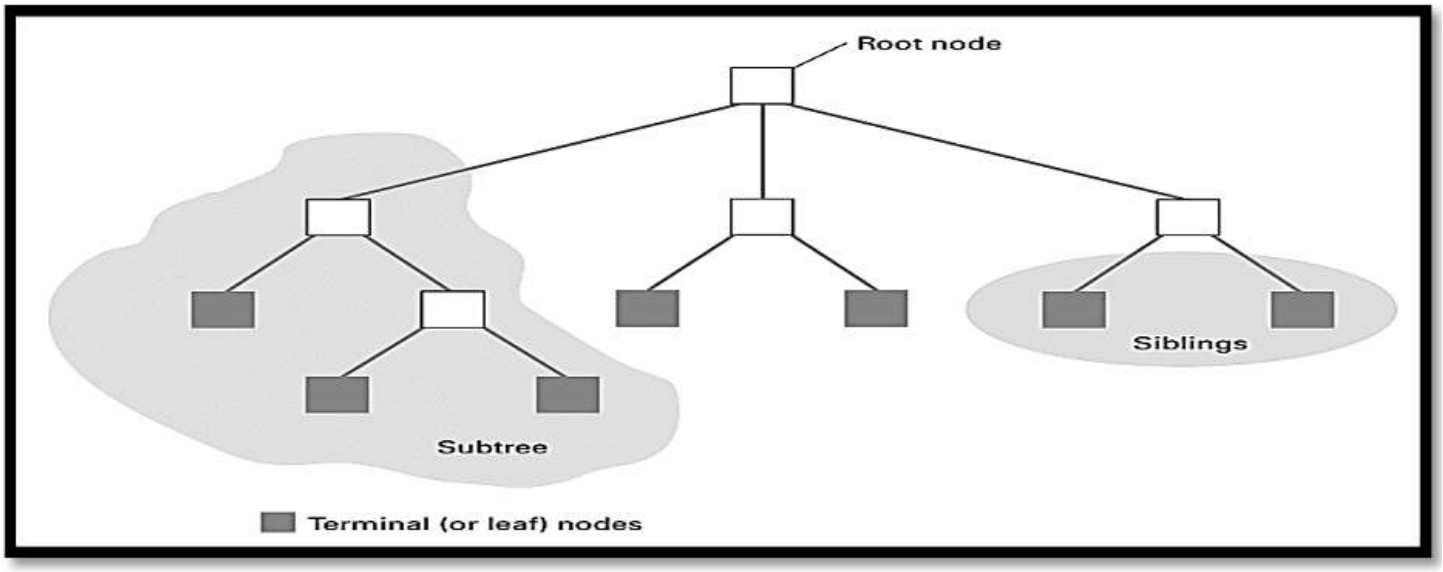
148. Data Abstraction: Trees

A **tree** is a collection whose entries have a hierarchical organization similar to that of an organization chart of a typical company



Each position in a tree is called a **Node**. The node at the top is called the **Root Node.** The nodes at the other extreme are called **terminal nodes** (or sometimes **leaf nodes**). We often refer to the number of nodes in the longest path from the root to a leaf as the **depth** of the tree. In other words, the depth of a tree is the number of horizontal layers within it.

Contact ME → Click Here



We refer to its immediate descendants as its **children** and its immediate ancestor as its parent. Moreover, we speak of nodes with the same parent as being **siblings**. A tree in which each parent has no more than two children is called a **binary tree.**

We call these smaller structures **subtrees.** Each such subtree is called a **branch** from the parent.

Module 149

149. Data Abstraction: Pointers

A **pointer** is a storage area that contains such an encoded address. In the case of data structures, pointers are used to record the location where data items are stored.

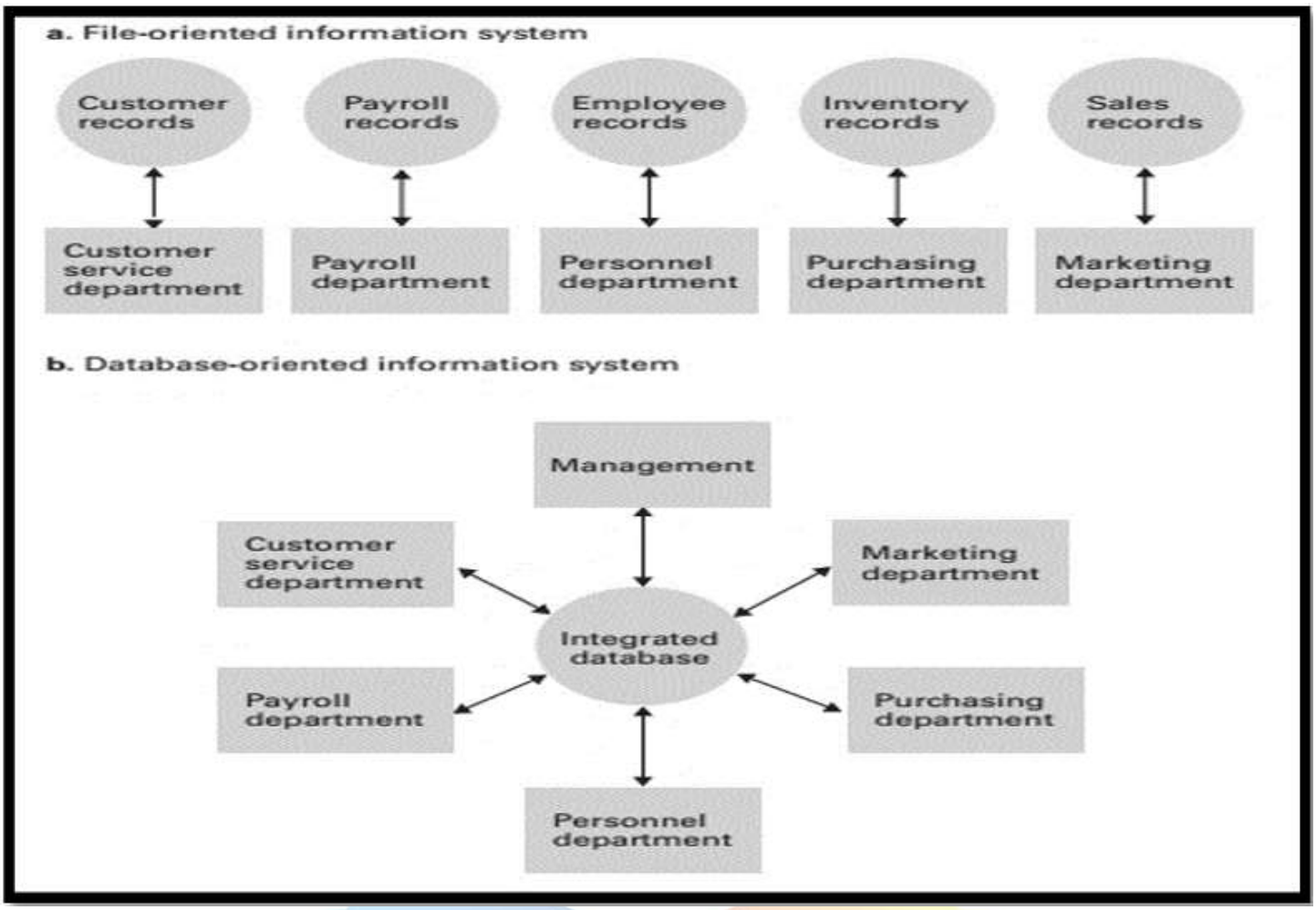
We have already encountered the concept of a pointer in our study of CPUs in the module 36. There we found that a register called a program counter is used to hold the address of the next instruction to be executed. Thus, the program counter plays the role of a pointer. In fact, another name for a program counter is **instruction pointer.**

Module 150

150. Database Systems: The Significance of Database Systems

The term **database** refers to a collection of data that is multidimensional in the sense that internal links between its entries make the information accessible from a variety of perspectives. This is in contrast to a traditional file system, sometimes called a **flat file,** which is a one-dimensional storage system, meaning that it presents its information from a single point of view.

Contact ME → Click Here



Module 151

151. Database Systems: Role of Schema

Learning & Growing With Team

A schema is a description of the entire database structure that is used by the database software to maintain the database.

A subschema is a description of only that portion of the database pertinent to a particular user's needs.

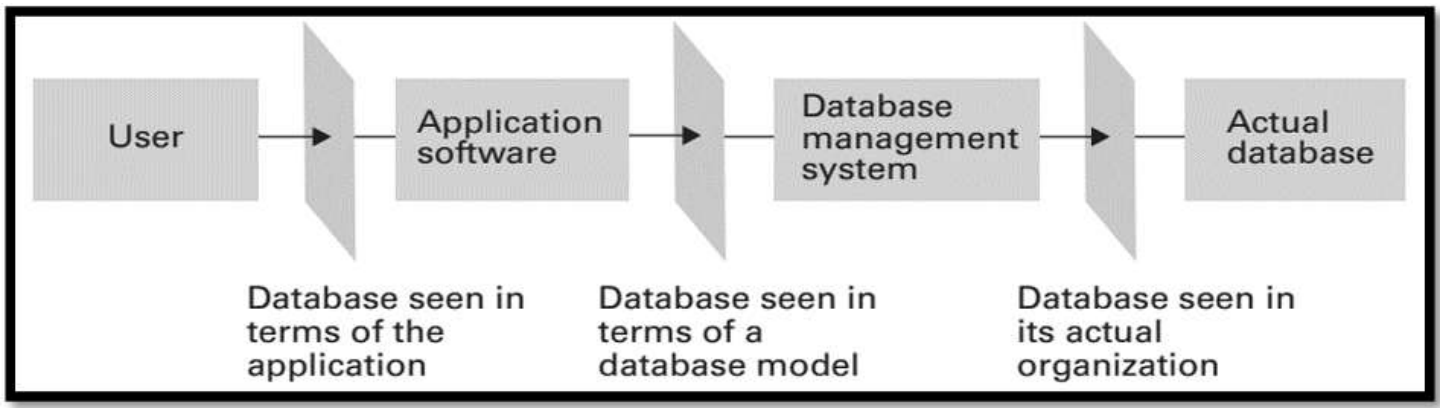
Module 152

152. Database Systems: Database Management Systems

A typical database application involves multiple software layers, which we will group into two major layers—**an application layer** and a **database management layer**

Software does not directly manipulate the data- base. The actual manipulation of the database is accomplished by the **database management system (DBMS).**

Contact ME → Click Here



Module 153

153. Database Systems: Relational Database model

A DBMS contains routines that translate commands stated in terms of a conceptual view of the database into the actions required by the actual data storage system. This conceptual view of the database is called a **database model.**

Relational database model portrays data as being stored in rectangular tables, called **relations,** which are similar to the format in which information is displayed by spreadsheet programs.

A row in a relation is called a **tuple** (some say "TOO-pul," others say "TUH-pul").

Columns in a relation are referred to as **attributes.**

Learning & Growing With Team

Module 156

156. Database Systems: Select Operation

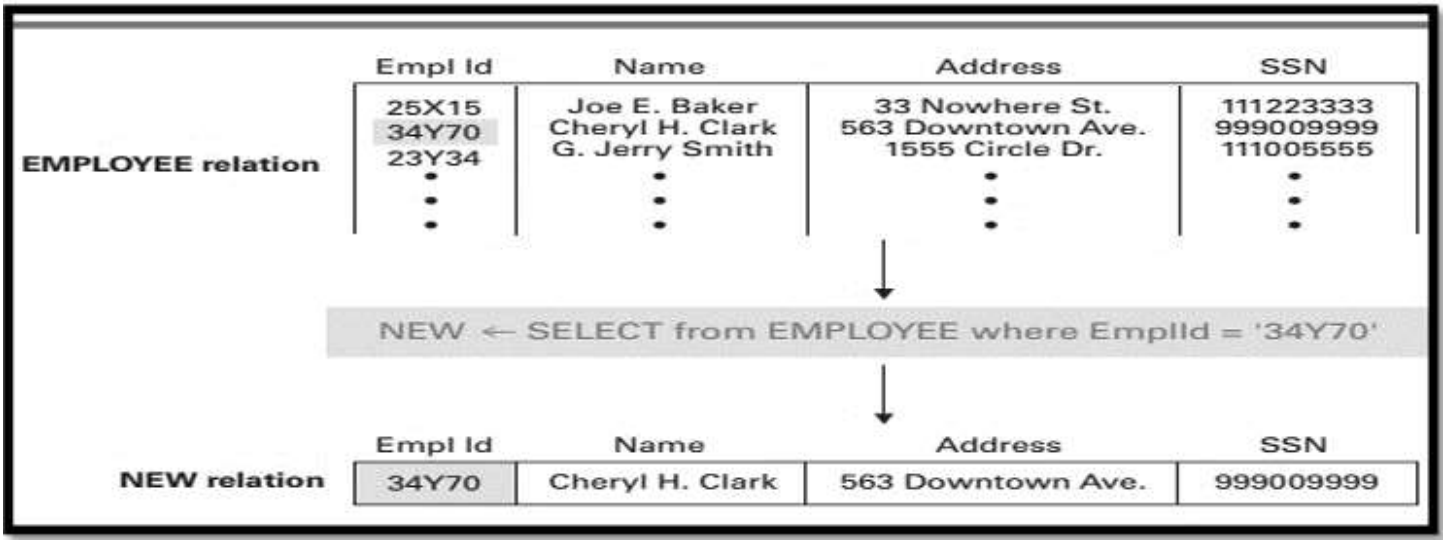
to select tuples possessing certain characteristics and to place these selected tuples in a new relation. To express this operation, we adopt the syntax

NEW <- SELECT from EMPLOYEE where EmplId = '34Y70'

The semantics of this statement is to create a new relation called **NEW containing**

Select Operator: Which Extract row from a relation or table

Contact ME → Click Here

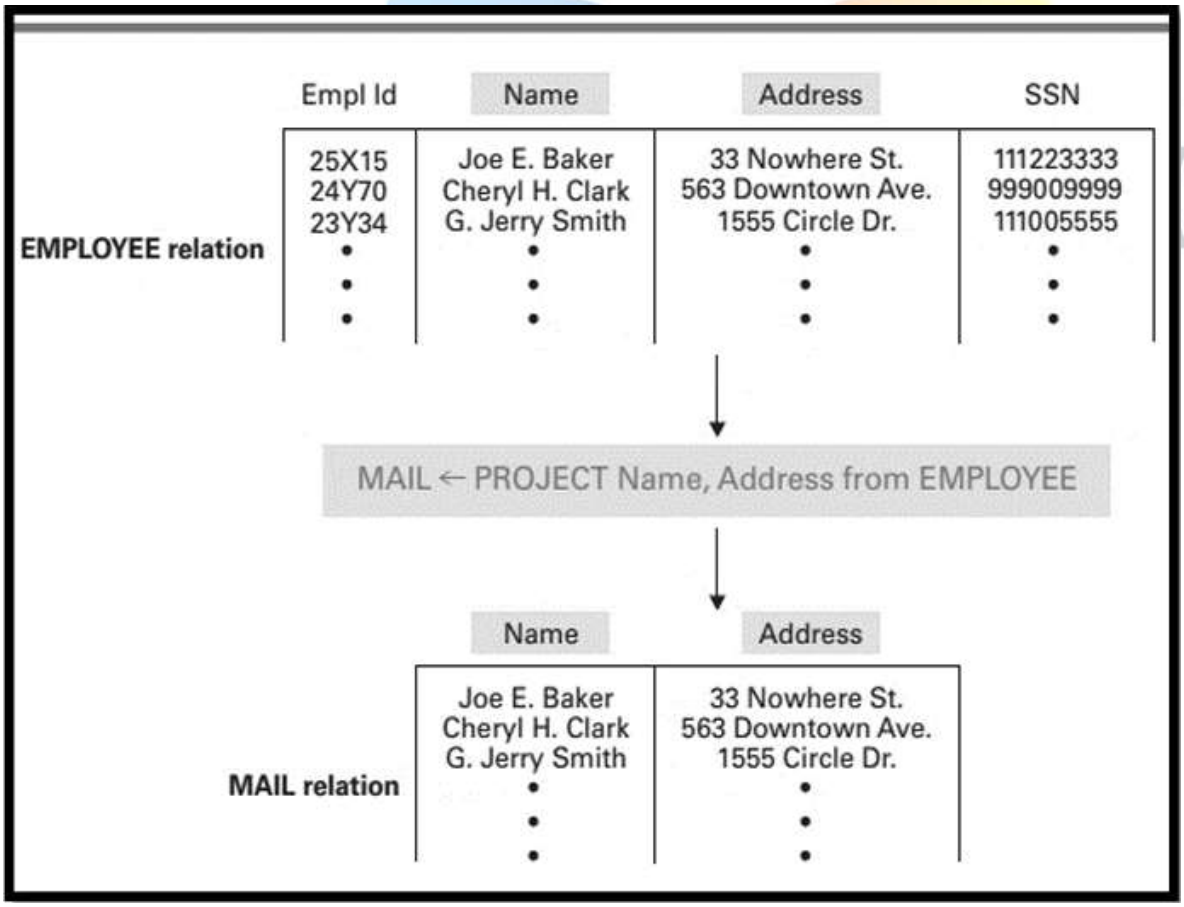


Module 157

157. Database Systems: project Operation

NEW2 <- PROJECT JobTitle from NEW1

MAIL <- PROJECT Name, Address from EMPLOYEE



Contact ME → Click Here

Module 161

161. Database Systems: The Commit/Rollback Protocol

A **single transaction**, such as the transfer of funds from one bank account to another.

the cancellation of an airline reservation, or the registration of a student in a university course, might involve multiple steps at the database level. ← It is just Explain not remember it

The point at which all the steps in a transaction have been recorded in the log is called the **commit point**.

The DBMS might find itself with a partially executed transaction that cannot be completed. In this case the log can be used to **roll back** *(undo)* the activities actually performed by the transaction.

The result is the problem known as **cascading rollback.**

Module 162

162. Database Systems: Locking

The **incorrect summary problem** can arise if one transaction is in the middle of transferring funds from one account to another when another transaction tries to compute the total deposits in the bank.

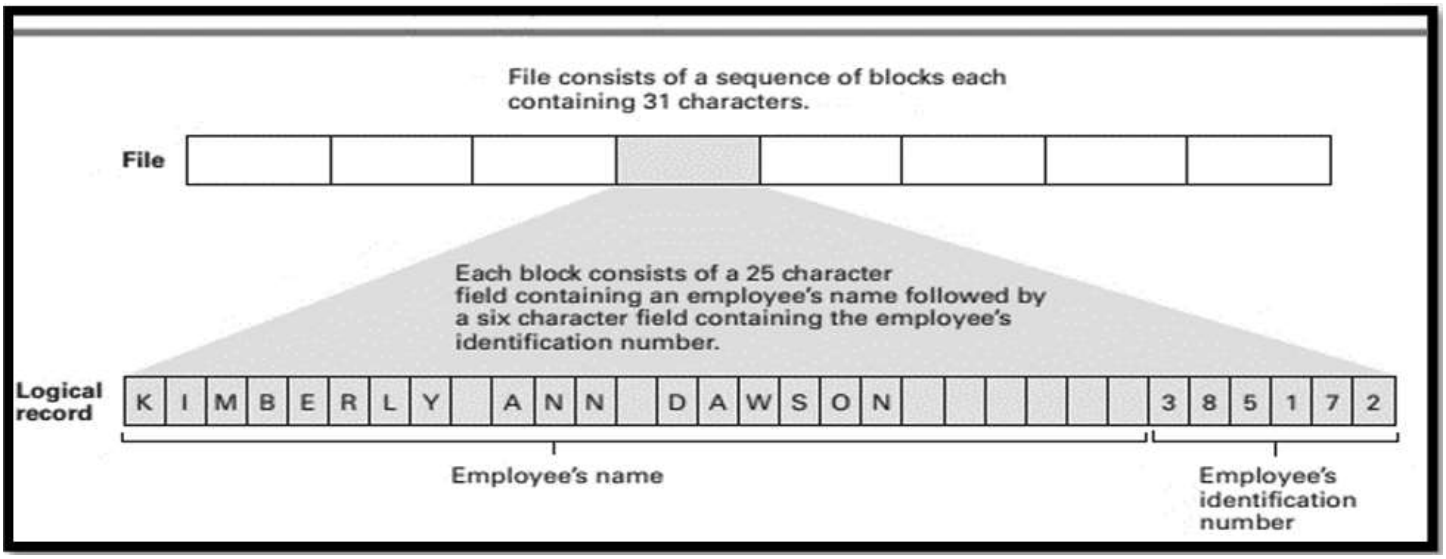
The **lost update problem**, which is exemplified by two transactions, each of which makes a deduction from the same account.

To guard against such anomalies as the incorrect summary problem and the lost update problem, these schedulers incorporate a **locking protocol** in which the items within a database that are currently being used by some transaction are marked as such. These marks are called locks; marked items are said to be locked. Two types of locks are common—**shared locks** and **exclusive locks.**

Module 163

163. Database Systems: Sequential Files

A **sequential file** is a file that is accessed in a serial manner from its beginning to its end as though the information in the file were arranged in one long row.



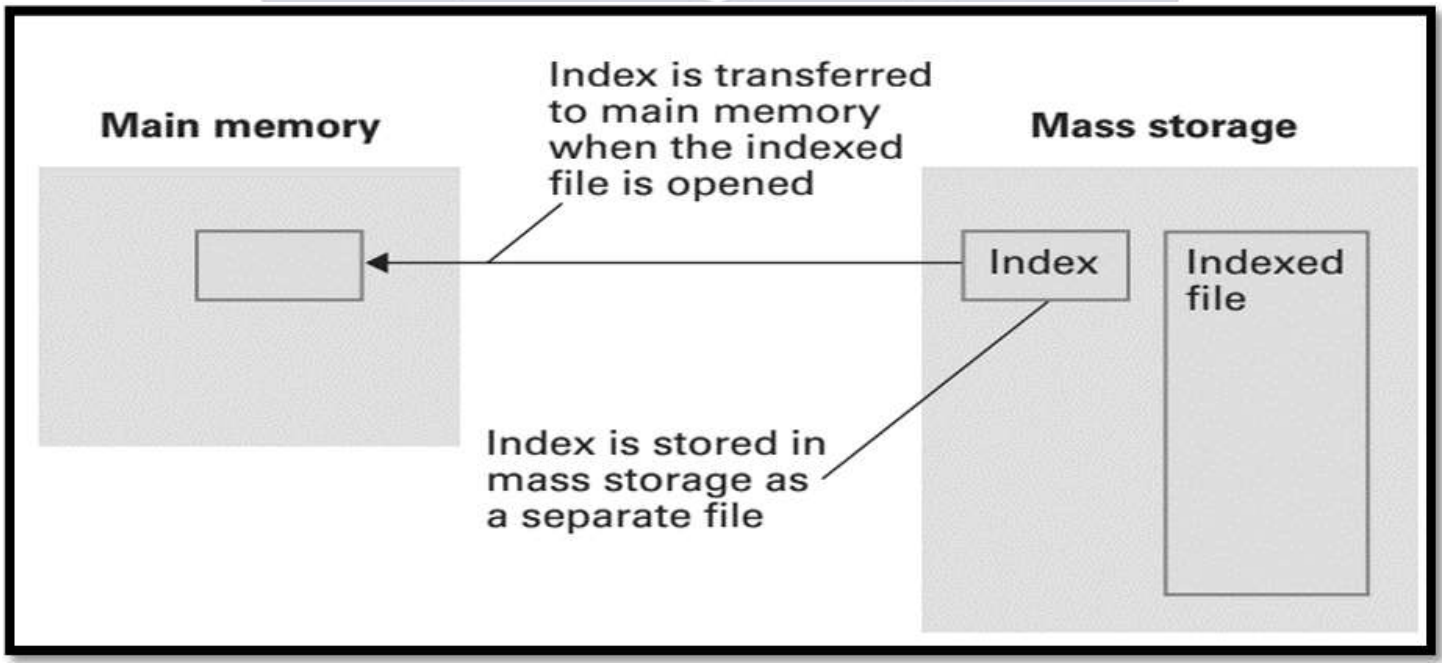
We refer to the end of a sequential file as the **end-of-file (EOF)**. There are a variety of ways of identifying the EOF. One is to place a special record, called a **sentinel.**

Module 164

164. Database Systems: Indexed Files

Means keh data ko store kaise kia jata hai us ko computer main rakhna kaise hai

A popular solution is to use an index for the file in much the same way that an index in a book is used to locate topics within the book. Such a file system is called an indexed file.



Contact ME → Click Here

Module 165

165. Database Systems: Hash Files

Hashing is a technique that provides similar access without such overhead

The data storage space is divided into several sections, called **buckets,** each of which is capable of holding several records.

This conversion from key values to bucket numbers is called a **hash function.**

When hashing is applied to a storage structure in mass storage, the result is called a **hash file.**

When applied to a storage structure within main memory, the result is usually called a **hash table.**

Module 167

167. Database Systems: Data Mining

Data mining, which consists of techniques for discovering patterns in collections of data. Data mining has become an important tool in numerous areas including marketing, inventory management, quality control, loan risk management, fraud detection, and investment analysis

Data mining is practiced on static data collections, called **data warehouses**, rather than "online" operational databases that are subject to frequent updates. These warehouses are often "snapshots" of databases or collections of databases.

Two common forms of data mining are **class description** and **class discrimination**.

Class description deals with identifying properties that characterize a given group of data items, whereas,

Class discrimination deals with identifying properties that divide two groups. whereas *class discrimination* techniques would be used to find proper- ties that distinguish customers who shop for used cars from those who shop for new ones.

Module 168

168. Database Systems: Data Mining Examples and Implications

Cluster analysis, which seeks to discover classes.

Data mining is **association analysis**, which involves looking for links between data groups.

Finally, the form of data mining called **sequential pattern analysis** tries to identify patterns of behavior over time.

Contact ME → Click Here

Data cubes (data viewed from multiple perspectives—the term cube is used to conjecture the image of multiple dimensions)

Module 169 are not important

Module 170

170. Artificial Intelligence: Introduction and Vision

Artificial intelligence is the field of computer science that seeks to build autonomous machines - machines that can carry out complex tasks without human intervention.



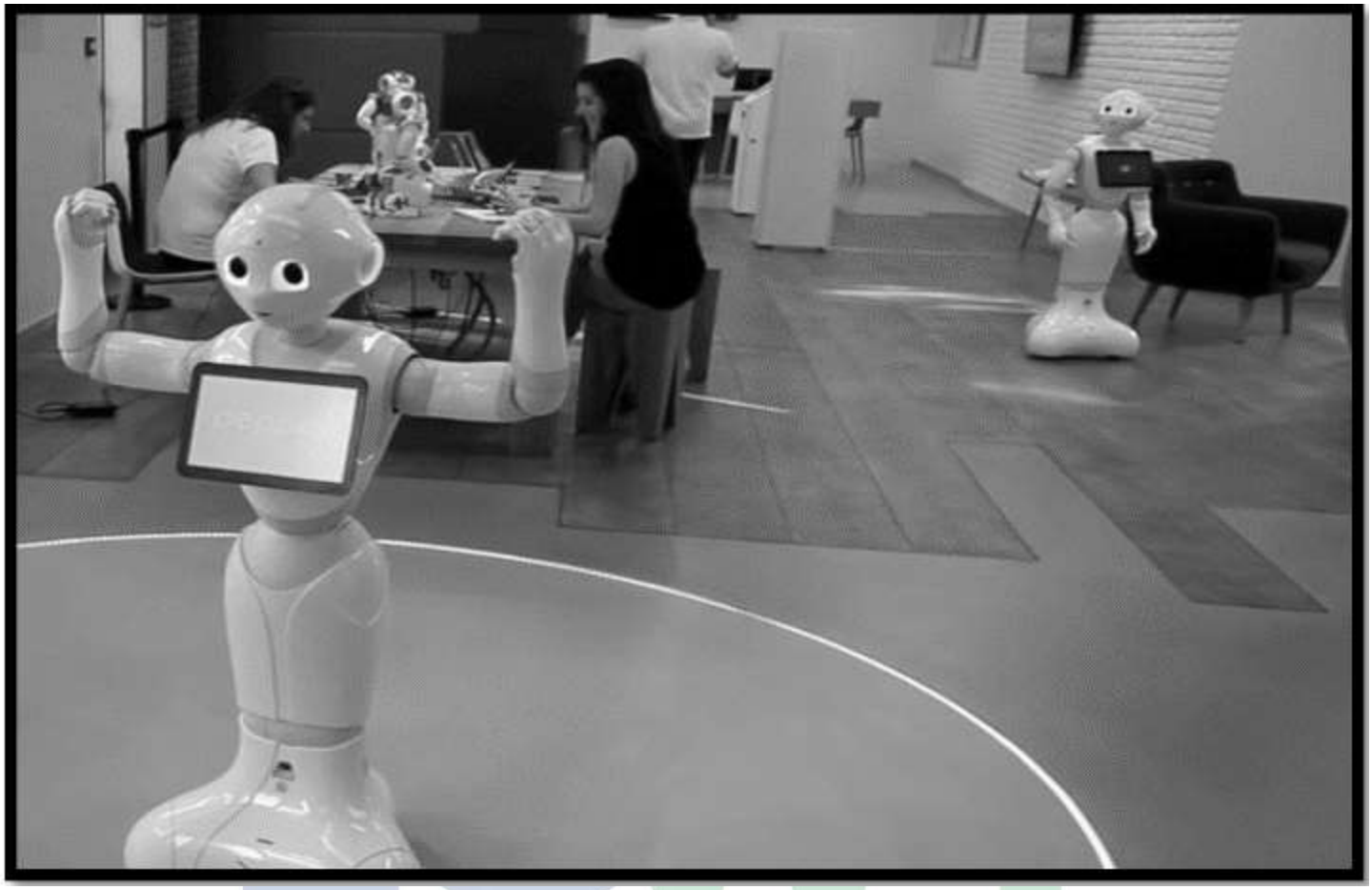
Robot



Self Drive Car

Learning & Growing With Team





receptionist in Belgian hospital

Module 171

171. Artificial Intelligence: Intelligent Agents

An agent is a “device” that responds to stimuli from its environment. It is natural to envision an agent as an individual machine such as a robot.

Examples of **sensors** include microphones, cameras, range sensors, and air or soil sampling devices.

Examples of **actuators** include wheels, legs, wings, grippers, and speech synthesizers.

procedural knowledge (learning “how”) or storing declarative knowledge (learning “what”).

Module 172

172. Artificial Intelligence: Research Methodologies

It is being pursued along **two** paths.

Contact ME → Click Here

- **Engineering track**
- **Theoretical track**

Engineering track in which researchers are trying to develop systems that exhibit intelligent behavior.

The engineering approach leads to a performance-oriented methodology because the underlying goal is to produce a product that meets certain performance goals.

The other is a **theoretical track** in which researchers are trying to develop a computational understanding of animal—especially human—intelligence.

The theoretical approach leads to a simulation-oriented methodology because the underlying goal is to expand our understanding of intelligence

Module 173

173. Artificial Intelligence: The Turing Test

Turing test (proposed by Alan Turing in 1950) has served as a benchmark in measuring progress in the field of artificial intelligence. Turing predicted that by the year 2000 machines would have a 30 percent chance of passing a five-minute Turing test.

The pro- gram DOCTOR (a version of the more general system called **ELIZA**) developed by Joseph Weizenbaum in the mid-1960s.

The computer played the role of the **analyst** while the user played the **patient**

Learning & Growing With Team

Module 176

176. CS Impact: CS impact on Society

Computer Science has impacted lives of humans largely. In this module we will learn the positive and negative impacts of CS on our society.

Positive impact of CS:

- ✓ It has improved the communication between people.
- ✓ Increased access to educational information via the Internet
- ✓ Increased productivity of people as now their manual work has reduced significantly.
- ✓ Exponential business growth
- ✓ Faster response times (for example, email, instant messaging and chat)

Negative Impact:

- ✓ People are working in sitting position and thus have more chance to develop obesity.
- ✓ Exposure to inappropriate material via the Internet

Contact ME → Click Here

- ✓ Increased crime and access to private information
- ✓ Hacking bank account and transfer money to your account
- ✓ People are now over-dependent on technology. They even try to catch the people online even if the person is sitting near them geographically. This has decreased the social interactions as well.

Module 177

177. CS Impact: CS Impact on Health

Computer Science has both positive and negative impacts on health.

Positive impact :

- ✓ Easy access to online health centers
- ✓ We can take online appointment
- ✓ We can receive reports via email at home
- ✓ Hospitals can be managed more efficiently
- ✓ Online health tips/issues are available.
- ✓ Health news can be viewed online.

Negative impact :

- ✓ Computer wrists – Typing too much. This creates issues in the hands.
- ✓ Eyesight – eye is not meant to look 2D for long amount of time. This increases the risk of eye
- ✓ diseases.
- ✓ Hearing loss is more common in those listening to high volume music on computer.
- ✓ Social Isolation
- ✓ Communication Issues
- ✓ Posture could lead bone issues
- ✓ Radiation Damage when the device is running. Its better to put off all such digital devices especially
- ✓ mobile phones or if it's real

Module 178

178. CS Impact: CS Impact on Environment

Computer Science has both positive and negative impacts on the environment as well.

- ✓ **Positive impact :**
- ✓ Paperless – Online reading and publishing.
- ✓ Americans use about 680 pounds of paper per person, per year
- ✓ This amounts to about 85 million tons of paper or 2 billion trees.
- ✓ The average American household throws away 13,000 pieces of paper (around 1 billion trees in total in USA) each year. Most in the form of packaging and junk mail.
- ✓ Due to the usage of computer, the above waste of trees is minimizing in the world.

Contact ME → Click Here

Negative impact :

- ✓ In offices, people print unnecessarily.
- ✓ Computers are made from heavy metals (Lead, mercury, Beryllium, Cadmium, PCV) when thrown away cause water contamination and air pollution.
- ✓ E-waste is sent to developing countries where people extract materials from these electronics such as gold, silver, and copper.
- ✓ They extract them by burning the substances and that process releases hazardous smoke into the
- ✓ air.
- ✓ The people present around the e-waste gain diseases such as skin cancer, brain damage, lung cancer, kidney disease
- ✓ Usage of energy has also increased with the use of computer.

Module 180

180. CS Impact: Software Licenses and Information Privacy

Software licenses typically provide end users with the right to one or more copies of the software without violating copyrights.

- **Information Privacy**

Data privacy, also called information privacy deals with the ability an organization or individual has to determine what data in a computer system can be shared with third parties.

- **Children's Online Privacy Protection Act**

(COPPA) - gives parents control over what information websites can collect from their kids.

- **Health Insurance Portability and Accountability Act**

(HIPPA) - ensures patient confidentiality for all healthcare-related data.

- **Electronic Communications Privacy Act**

ECPA extends government restrictions on wire taps to include transmissions of electronic data.

- **Video Privacy Protection Act**

Prevents wrongful disclosure of an individual's personally identifiable information stemming from their rental or purchase of audiovisual material.

- **Gramm-Leach-Bliley Act**

Mandates how financial institutions must deal with the private information of individuals.

Module 181

181. CS Impact: Intellectual Property

Contact ME → Click Here

Intellectual property (IP) is ideas, information and knowledge. Intellectual property rights (IPR) are specific legal rights which protect the owners of IP. IPR can be subdivided into the major categories

IPR	Covers	Need to apply?	Maximum duration
Patent	Inventions	Yes	20 years
Copyright	Literary, musical, artistic works, and software	No	70 years after death of author
Registered design	Image – look and feel	Yes	25 years
Registered trademark	Name, logo	Yes	Unlimited
Confidential information	Unpublished secret information	No	Unlimited
Database right	Databases	No	15 years

Contact ME → Click Here